CS459/659 Privacy, Cryptography, Network and Data Security

Privacy-Preserving Machine Learning

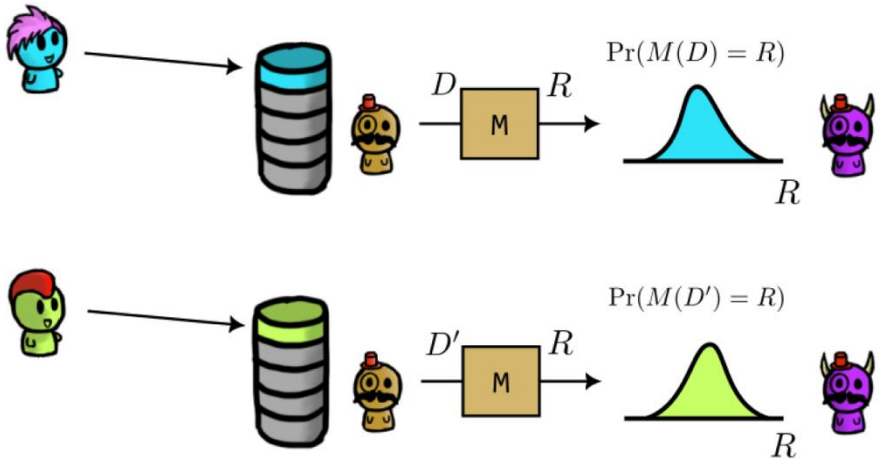
Spring 2026, Tuesday/Thursday 10:00-11:20am

Prologue: a few more DP properties

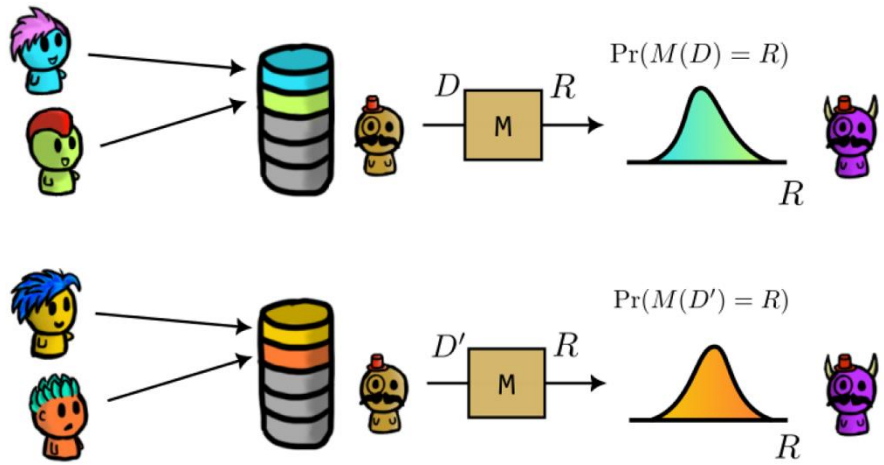
Recap on Group privacy

Group privacy: Let $M: \mathcal{D} \rightarrow \mathcal{R}$ be a mechanism that provides ϵ-DP for D, D' that differ in one entry. Then, it provides $k\epsilon$-DP for datasets D, D' that differ in k entries.

If this is ϵ-DP….



… then this is 2ϵ-DP



Group privacy with (ϵ, δ)-DP

- For approximate DP, δ gets an additional factor of $ke^{(k-1)\epsilon}$:

Group privacy: Let $M: \mathcal{D} \to \mathcal{R}$ be a mechanism that provides (ϵ, δ)-DP for D, D' that differ in one entry. Then, it provides $(k\epsilon, ke^{(k-1)\epsilon}\delta)$-DP for datasets D, D' that differ in k entries.

Sequential Composition

Naïve composition: Let $M = (M_1, M_2, \ldots, M_k)$ be a sequence of mechanisms, where M_i is (ϵ_i, δ_i)-DP. Then M is $(\sum_{i=1}^{k} \epsilon_i, \sum_{i=1}^{k} \delta_i)$-DP

- When running k mechanisms on the same sensitive dataset, and publishing all k results, the ϵs and δs **linearly** add up
  - Privacy decrease as we publish more results
  - i.e., more queries mean more leakage "linear sum of privacy losses"

- If we assume δ = 0, this boils down to "PURE" ϵ-Differential Privacy

Sequential Composition

Naïve composition: Let $M = (M_1, M_2, \ldots, M_k)$ be a sequence of mechanisms, where M_i is (ϵ_i, δ_i)-DP. Then M is $(\sum_{i=1}^{k} \epsilon_i, \sum_{i=1}^{k} \delta_i)$-DP

- When running k mechanisms on the same sensitive dataset, and publishing all k results, the ϵs and δs **linearly** add up
  - Privacy decrease as we publish more results
  - i.e., more queries mean more leakage "linear sum of losses"

Likely too pessimistic!

- If we assume δ = 0, this boils down to "PURE" ϵ-Differential Privacy

Sequential Composition

- If we allow the overall δ to be slightly larger ($\delta' > 0$), we can get a much smaller ϵ:

Advanced composition: Let $M = (M_1, M_2, \ldots, M_k)$ be a sequence of mechanisms, where M_i is (ϵ, δ)-DP.

Then M is $\left(\epsilon\sqrt{2k \cdot \ln\left(\frac{1}{\delta'}\right)} + \frac{k\epsilon(e^{\epsilon}-1)}{e^{\epsilon}+1}, k\delta + \delta'\right)$-DP

- This assumes that the "worst-case" privacy breach is very unlikely to happen in every single query simultaneously.

Sequential Composition

- If we allow the overall δ to be slightly larger (δ' > 0), we can get a much smaller ϵ:

Advanced composition: Let $M = (M_1, M_2, \ldots, M_k)$ be a sequence of [illegible]ms, where M_i is (ϵ, δ)-DP.

Then M is $\left(\epsilon\sqrt{2k \cdot \ln\left(\frac{1}{\delta'}\right)} + \frac{k\epsilon(e^{\epsilon}-1)}{e^{\epsilon}+1}, k\delta + \delta'\right)$-DP

A tighter analysis. Intense math.

- Note that the overall ϵ only grows on the order of $\sqrt{k}$ now (loosely speaking), and that if we allow higher δ' then we can get a smaller overall ϵ.

Sequential Composition

Advanced composition: Let $M = (M_1, M_2, \ldots, M_k)$ be a sequence of mechanisms, where M_i is (ϵ, δ)-DP.

Then M is $\left(\epsilon\sqrt{2k \cdot \ln\left(\frac{1}{\delta'}\right)} + \frac{k\epsilon(e^{\epsilon}-1)}{e^{\epsilon}+1}, k\delta + \delta'\right)$-DP

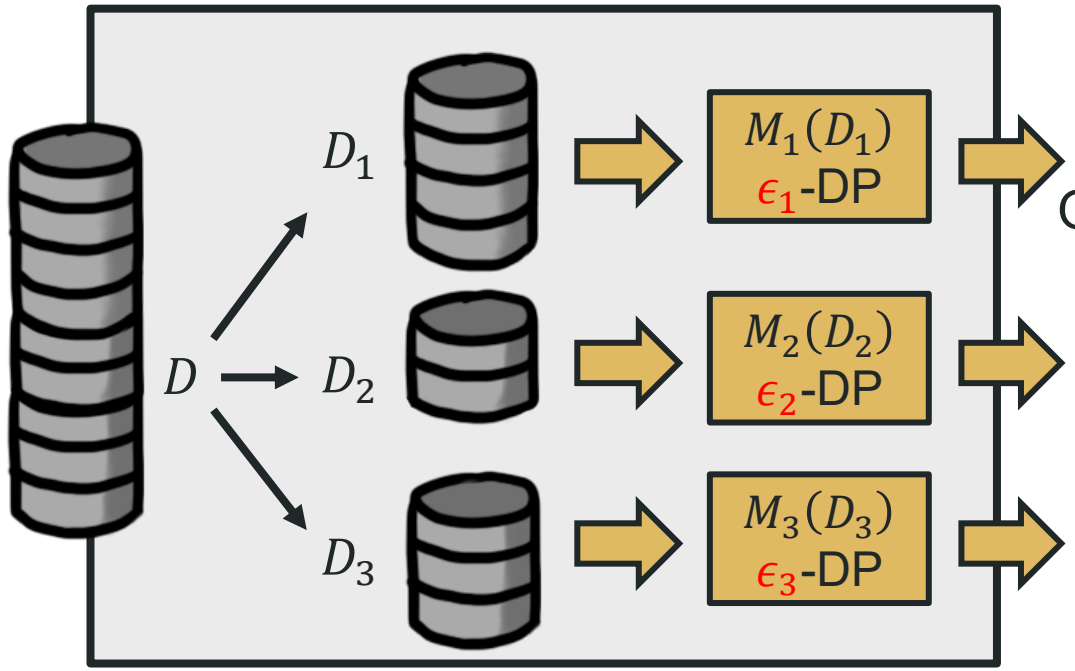
- Two goals:
  - $k\delta + \delta' \leq \frac{1}{N}$

Probability of privacy failure is no more than guessing the right individual from the dataset at random

  - δ' can be tuned to set trade-offs with ϵ "Better utility, with increased failure rate δ' ".

Parallel Composition

Parallel Composition: Let $M = (M_1, M_2, \ldots, M_k)$ be sequence of mechanisms, where M_i is ϵ_i-DP. Let $D_1, D_2, \ldots, D_k$ be a deterministic partition of D. Publishing $M_1(D_1), M_2(D_2), \ldots, M_k(D_k)$ satisfies $(\max_{i \in [1,\ldots,k]} \epsilon_i)$-DP.



Overall: $\max(\epsilon_1, \epsilon_2, \epsilon_3)$-DP



- The partition of D must be deterministic and have no overlaps
- i.e queries hit disjoint subsets

Renyi Differential Privacy

- Differential privacy is a very ambitious privacy guarantee, that protects against a **worst-case** adversary that potentially knows D and D', and for all possible outputs of the mechanism.

- Pure ϵ-DP and (ϵ, δ)-DP provide a very **limited** and **pessimistic** description of the differences between $\Pr(M(D) \in S)$ and $\Pr(M(D') \in S)$.

- There are other ***relaxed*** notions of DP that capture other nuances between these distributions, allowing for a tighter analysis.
  - Relaxes how much we care about the worst case (sometimes very unlikely)
  - A popular one is **Renyi Differential Privacy**

Renyi Differential Privacy

- Differential privacy is a very ambitious privacy guarantee, that protects against a **worst-case** adversary that potentially knows D and D', and for all possible outputs of the mechanism.

- Pure ϵ-DP and (ϵ, δ)-DP provide a very **limited** and **pessimistic** description of the differences between $\Pr(M(D) \in S)$ and $\Pr(M(D') \in S)$.

- There are other ***relaxed*** notions of DP that capture other nuances between these distributions, allowing for a tighter analysis.
  - Relaxes how much we care about the worst case (sometimes very unlikely)
  - A popular one is **Renyi Differential Privacy**

(α,ε)-Rényi Differential Privacy: $Div_{\alpha}(M(D), M(D')) \leq \epsilon$

    - $Div_{\alpha}(\ \ )$: Renyi Divergence; **α** changes how sensitive the divergence is to differences between the distributions.

Renyi Differential Privacy

- To introduce Renyi DP we need to know Renyi Divergence

Renyi Divergence: given two probability distributions P and Q, the Renyi divergence of order $\alpha > 1$ is

$$D_\alpha(P||Q) = \frac{1}{\alpha - 1} \log \mathbb{E}_{x \sim Q} \left(\frac{P(x)}{Q(x)} \right)^\alpha = \frac{1}{\alpha - 1} \log \left(\int_x P(x)^\alpha Q(x)^{1-\alpha} \, dx \right)$$

- The logarithm measures how much more or less likely an event x is under P compared to Q. (divergence is intended to measure how much information is lost when using Q(x) instead of P(x))

Renyi Differential Privacy

Renyi Divergence: given two probability distributions P and Q, the Renyi divergence of order $\alpha = \infty$ (defined by its limit) is

$$D_\infty(P||Q) = \sup_x \log\left(\frac{\mathrm{P}(x)}{Q(x)}\right) = \log\left(\max_x \frac{\mathrm{P}(x)}{Q(x)}\right)$$

Renyi Divergence: given two probability distributions P and Q, the Renyi divergence of order $\alpha = 1$ is the Kullback-Leibler divergence

$$D_1(P||Q) = \mathbb{E}_{x\sim P}\log\left(\frac{\mathrm{P}(x)}{Q(x)}\right) = \int_x P(x)\log\left(\frac{\mathrm{P}(x)}{Q(x)}\right)\,dx$$

(note that $x \sim P$ in the expectation)

Renyi Differential Privacy – DP connection

Renyi DP: a mechanism $M: \mathcal{D} \to \mathcal{R}$ is (ϵ, α)-RDP (also read as "ϵ-RDP of order α") if, for any neighboring datasets D, D' it holds that

$$D_\alpha(M(D) \| M(D')) \leq \epsilon$$

- Recall that, when $\alpha = \infty$, then the divergence (defined by its limit) is:

$$D_\infty(M(D) || M(D')) = \sup_x \log\left(\frac{\Pr(M(D) \in x)}{\Pr(M(D') \in x)}\right)$$

- In that case, it is easy to see that (ϵ, ∞)-RDP is equivalent to ϵ-DP

Renyi Differential Privacy: Properties

RDP Sequential Composition: if M_1 is (α, ϵ_1)-RDP and M_2 is (α, ϵ_2)-RDP, then the sequential composition (M_1, M_2) satisfies $(\alpha, \epsilon_1 + \epsilon_2)$-RDP

RDP to DP: if M is (α, ϵ)-RDP, then it is also $\left(\epsilon + \frac{\log\left(\frac{1}{\delta}\right)}{\alpha - 1}, \delta\right)$-DP

Renyi Differential Privacy: Example

Let's consider the following probability distributions:

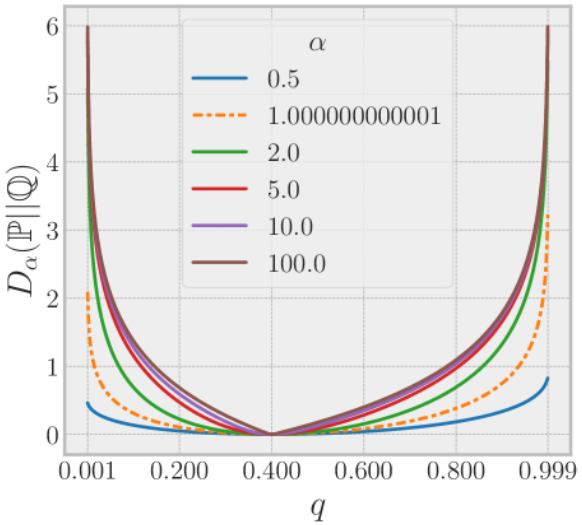
- $P=\{P(x_1) = 0.6, P(x_2) = 0.4\}$; $Q=\{Q(x_1) = 0.5, Q(x_2) = 0.5\}$

For $\alpha = 1$ (KL Divergence):

For $\alpha > 1$ (e.g., $\alpha=2$):

For $\alpha = \infty$:

Renyi Differential Privacy: Example



Let's consider the following probability distributions:

- $P=\{P(x_1)=0.6, P(x_2)=0.4\}$; $Q=\{Q(x_1)=0.5, Q(x_2)=0.5\}$

For $\alpha = 1$ (KL Divergence):

$$D_1(P||Q) = P(x_1)\log\left(\frac{P(x_1)}{Q(x_1)}\right) + P(x_2)\log\left(\frac{P(x_2)}{Q(x_2)}\right) \approx 0.1092 - 0.0892 = 0.0202$$

For $\alpha > 1$ (e.g., $\alpha=2$):

$$D_2(P||Q) = \frac{1}{2-1}\log \quad (P(x_1)^2\, Q(x_1)^{-1} + P(x_2)^2\, Q(x_2)^{-1}) \quad = \log(0.36 \cdot 2 + 0.16 \cdot 2) \approx 0.0392.$$

For $\alpha = \infty$ (Max Divergence):

$$D_\infty(P||Q) = \log\left(\max_x \frac{P(x)}{Q(x)}\right) \approx 0.1823 \quad \text{For } x_1 : \left(\frac{0.6}{0.5}\right) = 1.2 \quad \text{For } x_2 : \left(\frac{0.4}{0.5}\right) = 0.8$$

Many other variations of DP…

- An SOK from 2020

Name & references
(ε, δ)-approximate DP [52]
(ε, δ)-probabilistic DP [20, 124, 127]
ε-Kullback-Leiber Pr [9, 31]
(α, ε)-Rényi DP [128]
ε-mutual-information DP [31]
(μ, τ)-mean concentrated DP [58]
(ξ, ρ)-zero concentrated DP [19]
(f, ε)-divergence DP [9]
ε-unbounded DP [105]
ε-bounded/attribute/bit DP [105]
(c, ε)-group DP [49]
ε-free lunch Pr [105]
(R, c, ε)-dependent DP [116]
(P, ε)-one-sided DP [42]
(D, ε)-individual DP [149]
(D, t, ε)-per-instance DP [162]
$(\mathcal{R}, \varepsilon)$-generic DP [105]
$(G, \mathcal{I}_Q, \varepsilon)$-blowfish Pr [84, 86]
ε-adjacency-relation div. DP [97]
Ψ-personalized DP [59, 76, 94, 118]
Ψ-tailored DP/$\varepsilon(\cdot)$-outlier Pr [120]
$(\pi, \gamma, \varepsilon)$-random DP [83]
$d_{\mathcal{D}}$-Pr [22]
(ε, γ)-distributional Pr [141, 177]
$(\varepsilon(\cdot), \delta(\cdot))$-endogenous DP [107]
$(d_{\mathcal{D}}, \varepsilon, \delta)$-pseudo-metric DP [36]
$(\theta, \varepsilon, \gamma, \delta)$-typical Pr [10]
(Θ, ε)-on average KL Pr [164]
(f, d, ε)-extended divergence DP [97]
$(\mathcal{R}, M)$-general DP [103]
(Θ, ε)-noiseless Pr [14, 44]
(Θ, ε)-distributional DP [11, 35]
$(\Theta, \varepsilon, \delta)$-active PK DP [11, 14, 35]
$(\Theta, \varepsilon, \delta)$-passive PK DP [35]
$(\Theta, \Phi, \varepsilon)$-pufferfish Pr [106]
$(\Theta, \varepsilon, \delta)$-distribution Pr [98]
(d, Θ, ε)-extended DnPr [98]
(f, Θ, ε)-divergence DnPr [97]
$(d, f, \Theta, \varepsilon)$-ext. div. DnPr [97]
(Θ, ε)-positive membership Pr [114]
$(\Theta, \varepsilon, \delta)$-adversarial Pr [139]
(Θ, ε)-aposteriori noiseless Pr [14]
ε-semantic Pr [69, 96]
$(\mathsf{Agg}, \varepsilon)$-zero-knowledge Pr [72]
$(\Theta, \Gamma, \varepsilon)$-coupled-worlds Pr [11]
$(\Theta, \Gamma, \varepsilon, \delta)$-inference-based CW Pr [11]
ε_κ-SIM-computational DP [129]
ε_κ-IND-computational DP [129]
$(\mathsf{Agg}, \varepsilon)$-computational ZK Pr [72]

A noise mechanism for (ϵ, δ)-DP

The Gaussian Mechanism

- So far, we have seen a mechanism (Laplace) for pure DP. Let's see one for approximate DP.
- First, given a function $f: \mathcal{D} \to \mathbb{R}^k$, we define the ℓ_2-sensitivity as:

$$\Delta_2 \doteq \max_{D,D'} ||f(D) - f(D')||_2$$

l_2 norm: $\|x\|_2 = sqrt(x_1^2 + x_2^2 + \cdots + x_n^2)$

The Gaussian Mechanism

Similar to Laplace mechanism. Key differences:
Δ_2 instead of Δ_1
Adds Gaussian noise instead (still 0 mean)
The variance of the noise σ^2 is more complicated

- Given a function $f: \mathcal{D} \to \mathbb{R}^k$, we define the ℓ_2-sensitivity as:

$$\Delta_2 \doteq \max_{D,D'} ||f(D) - f(D')||_2$$

- The Gaussian mechanism simply adds Gaussian noise to the output of the function:

Given a function $f: \mathcal{D} \to \mathbb{R}^k$ with ℓ_2-sensitivity Δ_2, the **Gaussian mechanism** is defined as $M(D) = f(D) + (Y_1, Y_2, \ldots, Y_k)$ where each Y_i is random noise independently distributed as $\mathrm{Y_i} \sim N(0, \sigma^2)$ with $\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2$

The Gaussian Mechanism

- Given a function $f: \mathcal{D} \rightarrow \mathbb{R}^k$, we define the ℓ_2-sensitivity as:

$$\Delta_2 \doteq \max_{D,D'} ||f(D) - f(D')||_2$$

- The Gaussian mechanism simply adds Gaussian noise to the output of the function:

Given a function $f: \mathcal{D} \rightarrow \mathbb{R}^k$ with ℓ_2-sensitivity Δ_2, the **Gaussian me**
is defined as $M(D) = f(D) + (Y_1, Y_2, \ldots, Y_k)$ where each Y_i is ran
independently distributed as $\mathrm{Y_i} \sim N(0, \sigma^2)$ with $\sigma^2 = 2 \ln$

The Gaussian mechanism provides ϵ, δ-DP

Let's think about this

The Gaussian mechanism $M(D) = f(D) + Y$ where $Y \sim N(0, \sigma^2)$ with $\sigma^2 = 2 \ln\left(\frac{1.25}{\delta}\right) \Delta_2^2 / \epsilon^2$ provides (ϵ, δ)-DP.

Laplace noise gives strong, strict guarantees about privacy (ϵ).

Gaussian noise is more flexible, ensures decent privacy for the average population, but it is a bit "fuzzier" on the edges (δ)

Let's think about this

The Gaussian mechanism $M(D) = f(D) + Y$ where $Y \sim N(0, \sigma^2)$ with $\sigma^2 = 2 \ln\left(\frac{1.25}{\delta}\right) \Delta_2^2 / \epsilon^2$ provides (ϵ, δ)-DP.

Q: does the relationship between the privacy parameter ϵ and the noise variance σ^2 make sense?

Let's think about this

The Gaussian mechanism $M(D) = f(D) + Y$ where $Y \sim N(0, \sigma^2)$ with $\sigma^2 = 2 \ln\left(\frac{1.25}{\delta}\right) \Delta_2^2 / \epsilon^2$ provides (ϵ, δ)-DP.

Q: does the relationship between the privacy parameter ϵ and the noise variance σ^2 make sense?

A: yes, to provide **more privacy** (lower ϵ) we need **more noise** (higher σ^2).

Let's think about this

The Gaussian mechanism $M(D) = f(D) + Y$ where $Y \sim N(0, \sigma^2)$ with $\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2$ provides (ϵ, δ)-DP.

Q: if we fix the noise level (σ), what is the relationship between ϵ and δ, and why?

Let's think about this

The Gaussian mechanism $M(D) = f(D) + Y$ where $Y \sim N(0, \sigma^2)$ with $\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2$ provides (ϵ, δ)-DP.

Q: if we fix the noise level (σ), what is the relationship between ϵ and δ, and why?

A: for a fixed noise, ϵ and δ will be **inversely proportional**: if we want to allow for a higher δ, then that level of noise can provide lower ϵ's.

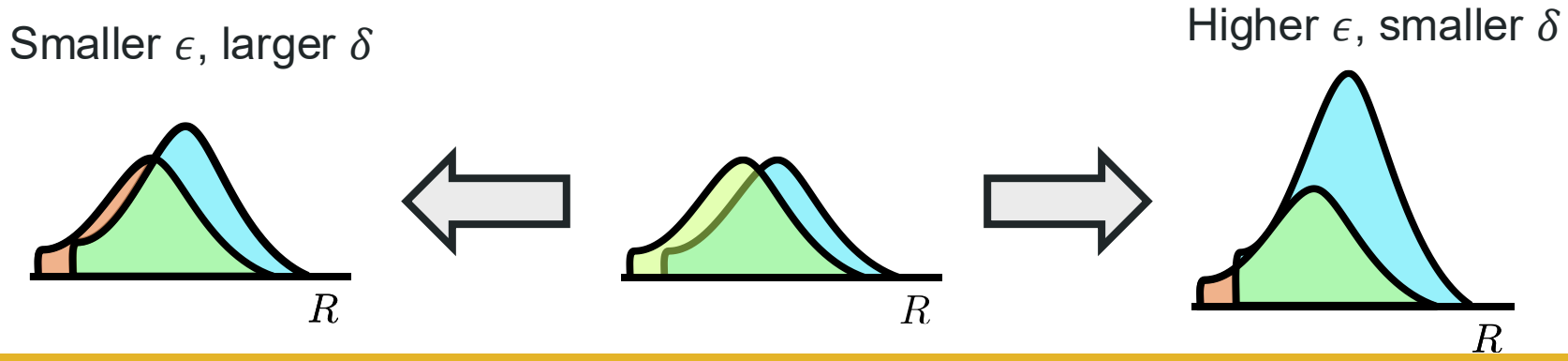
Let's think about this

The Gaussian mechanism $M(D) = f(D) + Y$ where $Y \sim N(0, \sigma^2)$ with $\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2$ provides (ϵ, δ)-DP.

Q: if we fix the noise level (σ), what is the relationship between ϵ and δ, and why?

A: for a fixed noise, ϵ and δ will be **inversely proportional**: if we want to allow for a higher δ, then that level of noise can provide lower ϵ's.

This is not just for the Gaussian mechanism, but all ϵ, δ-DP mechanisms:



Gaussian Mechanism: examples

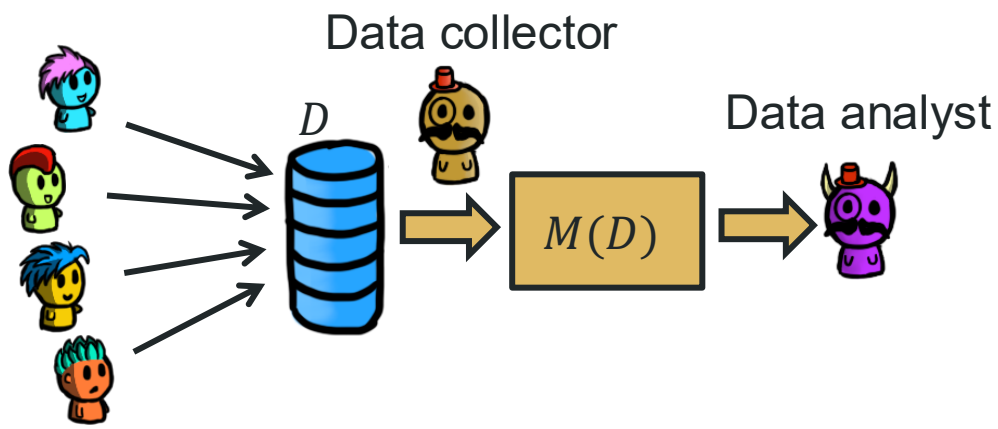
Example 1: D contains the salaries of a set of n users. The salaries range from 10k to 200k. We want to release the **total** salary of the users. What is the noise variance σ^2 of the gaussian mechanism under bounded DP assuming $\delta = 1/n^2$

$$\Delta_2 \doteq \max_{D,D'} ||f(D) - f(D')||_2$$

$f(D) + Y$ is (ϵ, δ)-DP if

$$Y \sim N(0, \sigma^2)$$

$$\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2$$



Gaussian Mechanism: examples

$$\Delta_2 \doteq \max_{D,D'} ||f(D) - f(D')||_2$$

Example 1: D contains the salaries of a set of n users. The salaries range from 10k to 200k. We want to release the **total** salary of the users. What is the noise variance σ^2 of the gaussian mechanism under bounded DP assuming $\delta = 1/n^2$

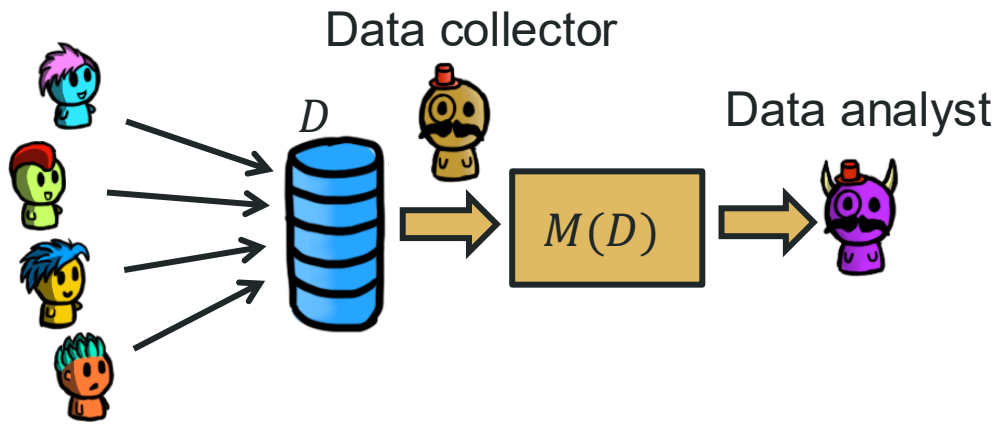
$f(D) + Y$ is (ϵ, δ)-DP if

$Y \sim N(0, \sigma^2)$

$$\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2$$

A: sensitivity is 190k

$$\sigma^2 = 2\ln(1.25\, n^2)(190k)^2/\epsilon^2$$



Gaussian Mechanism: examples

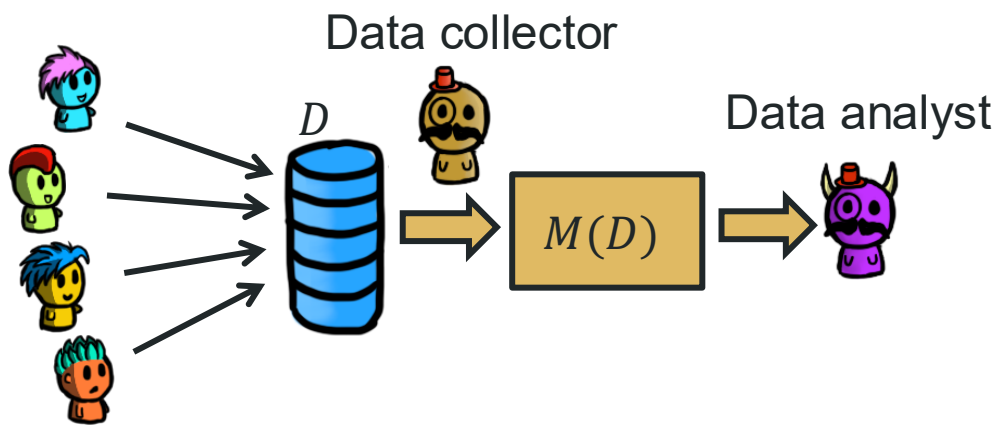
Example 2: D contains the age of a set of users. We want to release the histogram of ages [0-10), [10-20)…[100,110). What is the σ^2 of the gaussian mechanism under bounded DP assuming $\delta = 1/n^2$

$$\Delta_2 \doteq \max_{D,D'} ||f(D) - f(D')||_2$$

$f(D) + Y$ is (ϵ, δ)-DP if

$$Y \sim N(0, \sigma^2)$$

$$\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2$$



Gaussian Mechanism: examples

$$\Delta_2 \doteq \max_{D,D'} ||f(D) - f(D')||_2$$

Example 2: D contains the age of a set of users. We want to release the histogram of ages [0-10), [10-20)…[100,110). What is the σ^2 of the gaussian mechanism under bounded DP assuming $\delta = 1/n^2$

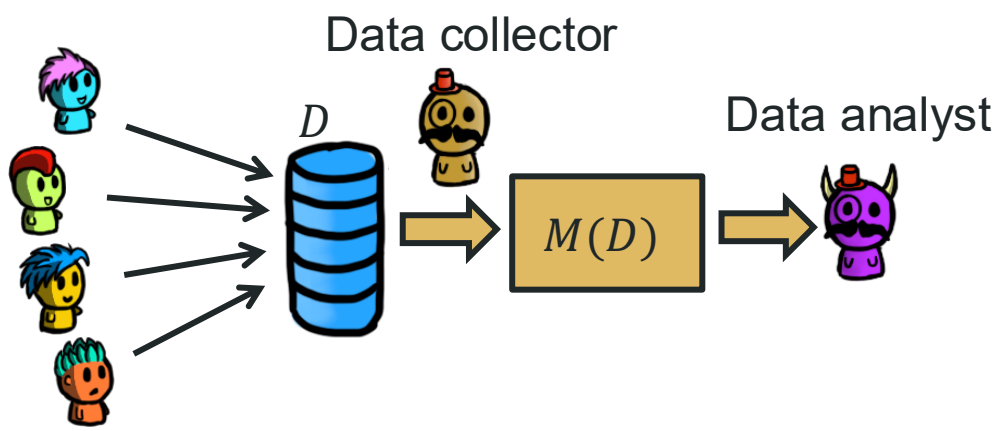
$f(D) + Y$ is (ϵ, δ)-DP if

$Y \sim N(0, \sigma^2)$

$$\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2$$

A: sensitivity $\sqrt{2}$ in bounded DP

$$\sigma^2 = 4\ln(1.25\, n^2)/\epsilon^2$$



Primer on Machine Learning

Machine learning: quick primer

- For simplicity, we will focus on a *classification problem* with *supervised learning*.
  - Unsupervised or Reinforcement learning are other types
- We have a training set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ with n samples. Given a sample (x_i, y_i), x_i are the *features* and y_i is its *label*.

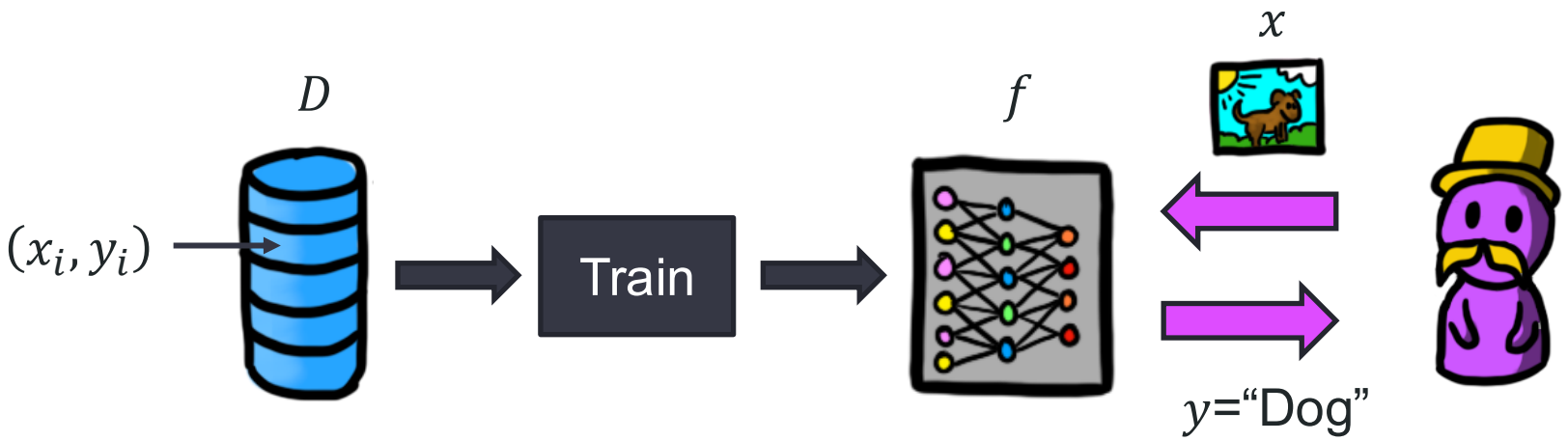
Machine learning: quick primer

- For simplicity, we will focus on a *classification problem* with *supervised learning*.
  - Unsupervised or Reinforcement learning are other types
- We have a training set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ with n samples. Given a sample (x_i, y_i), x_i are the *features* and y_i is its *label*.
- We want to produce a function $f: \mathcal{X} \to \mathcal{Y}$ that can predict a sample's label from its features.

Machine learning: quick primer

- For simplicity, we will focus on a *classification problem* with *supervised learning*.
  - Unsupervised or Reinforcement learning are other types
- We have a training set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ with n samples. Given a sample (x_i, y_i), x_i are the *features* and y_i is its *label*.
- We want to produce a function $f: \mathcal{X} \to \mathcal{Y}$ that can predict a sample's label from its features.
- We will use the training set to train such a function. Ideally, it should correctly predict labels for unseen samples (e.g., samples in a testing set).
  - We will say that a model *generalizes* well if it has high accuracy on unseen samples
  - A model *overfits* if it works perfectly for samples in the training set but does not generalize.

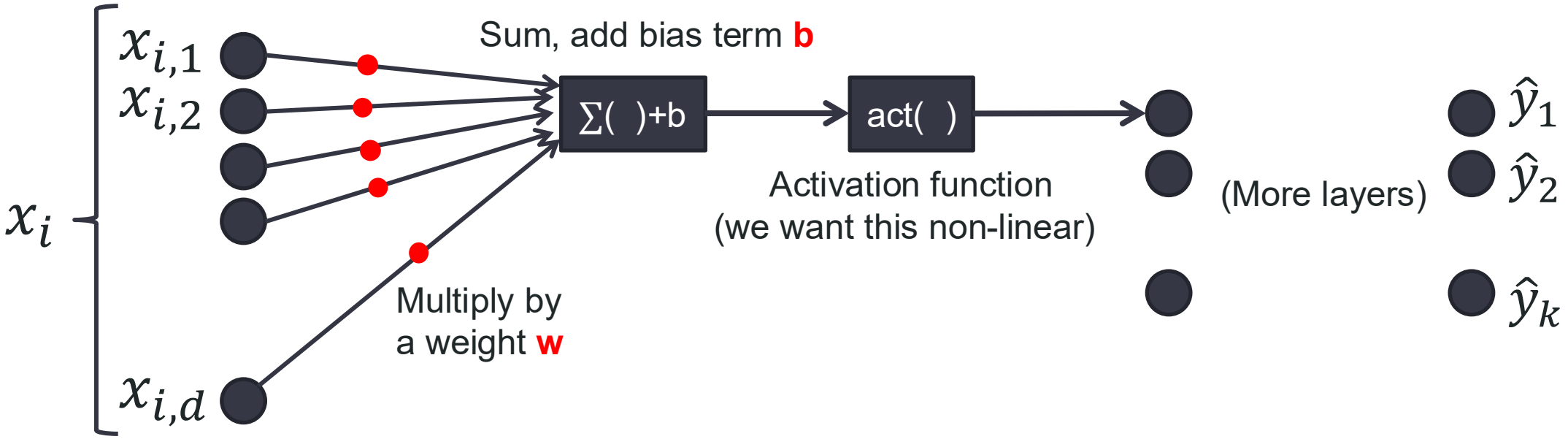
Machine learning: quick primer



Usually, this gives confidence scores for each class: $(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_k)$
For example: ["Dog", "Cat", "Mouse" …]=[0.81, 0.10, 0.03, …]

Neural networks

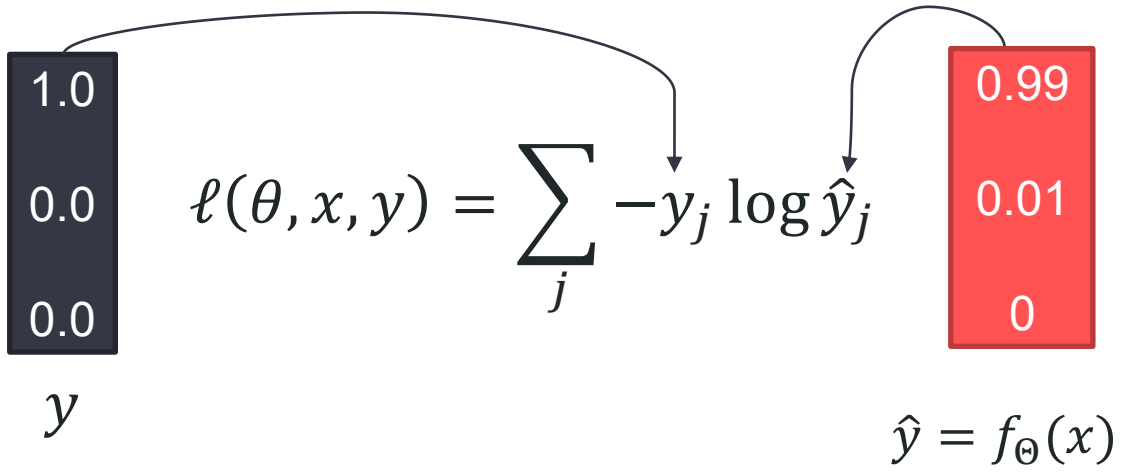
- There are many architectures for machine learning models (i.e., many structures for the function f).
- One of the most popular are **neural networks**.



Training the model means tuning all **w**'s and **b**'s

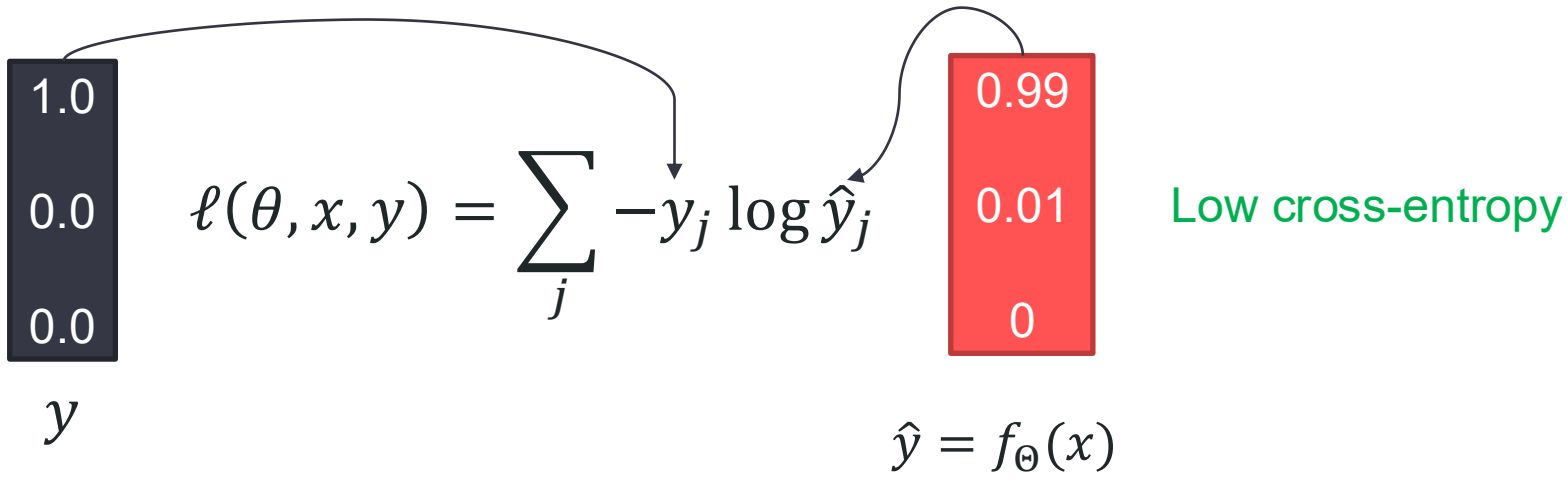
Loss Functions

- We define a *loss function* that we want to minimize: $\ell(\theta, x, y)$, where $\boldsymbol{\theta}$ are the parameters w and b.
  - E.g., **Softmax Cross-Entropy Loss**
    - $\ell(\theta, x, y) = \sum_j -y_j \log \hat{y}_j$, where y_j is only 1 for the true label of the sample j.
      $\hat{y}_j$ is the predicted probability "model output" for class j



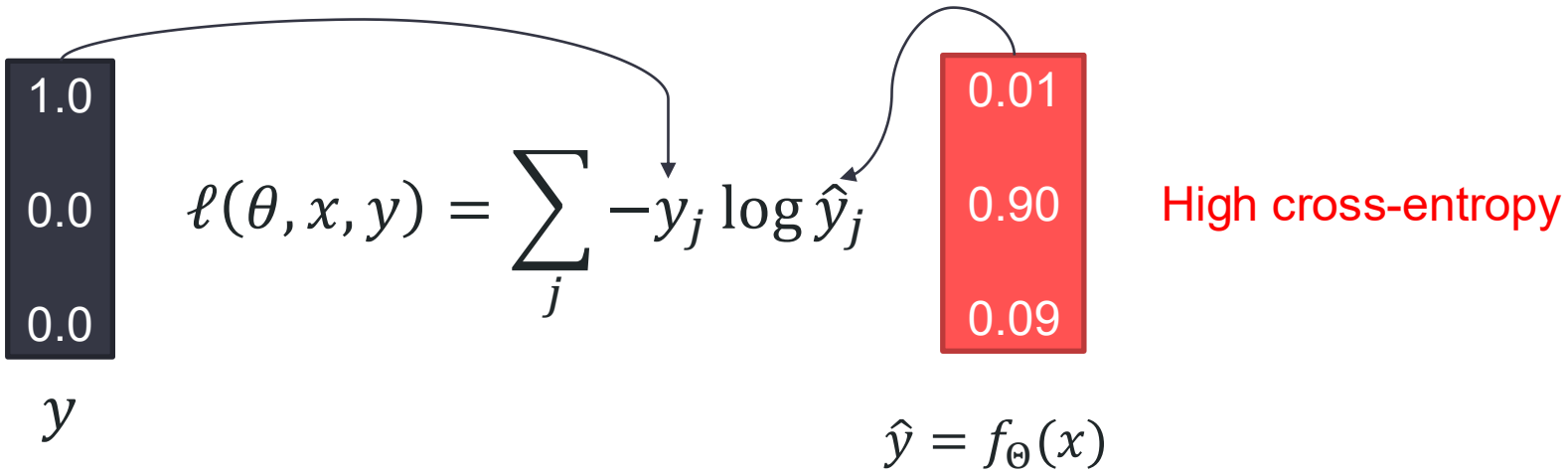
Loss Functions

- We define a *loss function* that we want to minimize: $\ell(\theta, x, y)$, where $\boldsymbol{\theta}$ are the parameters w and b.
  - E.g., **Softmax Cross-Entropy Loss**
    - $\ell(\theta, x, y) = \sum_j -y_j \log \hat{y}_j$, where y_j is only 1 for the true label of the sample j.
      $\hat{y}_j$ is the predicted probability "model output" for class j



Loss Functions

- We define a *loss function* that we want to minimize: $\ell(\theta, x, y)$, where $\boldsymbol{\theta}$ are the parameters w and b.
  - E.g., **Softmax Cross-Entropy Loss**
    - $\ell(\theta, x, y) = \sum_j -y_j \log \hat{y}_j$, where y_j is only 1 for the true label of the sample j.
      $\hat{y}_j$ is the predicted probability "model output" for class j

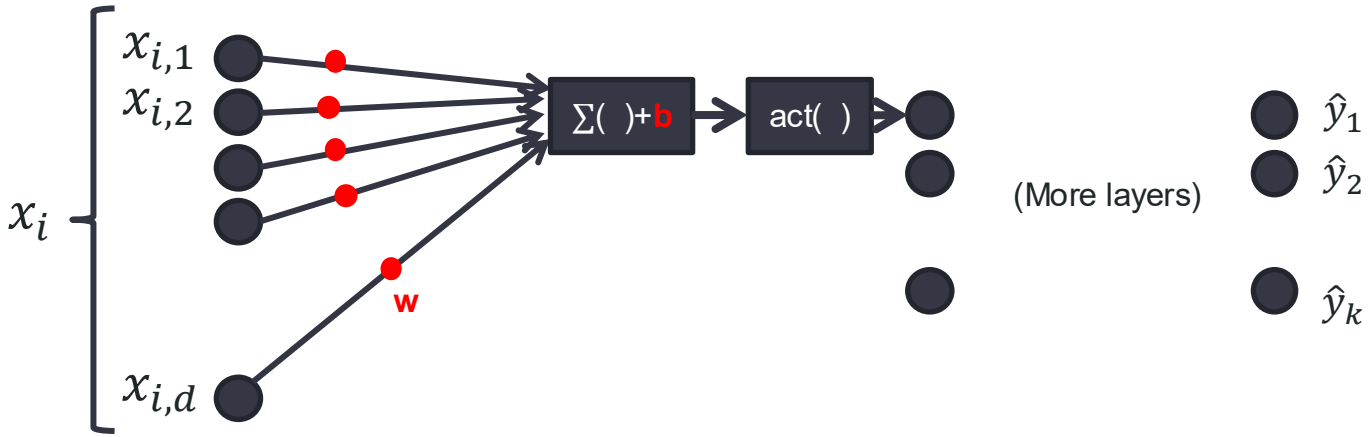


Training neural networks

- Since we have the training set D, it makes sense to minimize the empirical loss in this training set:

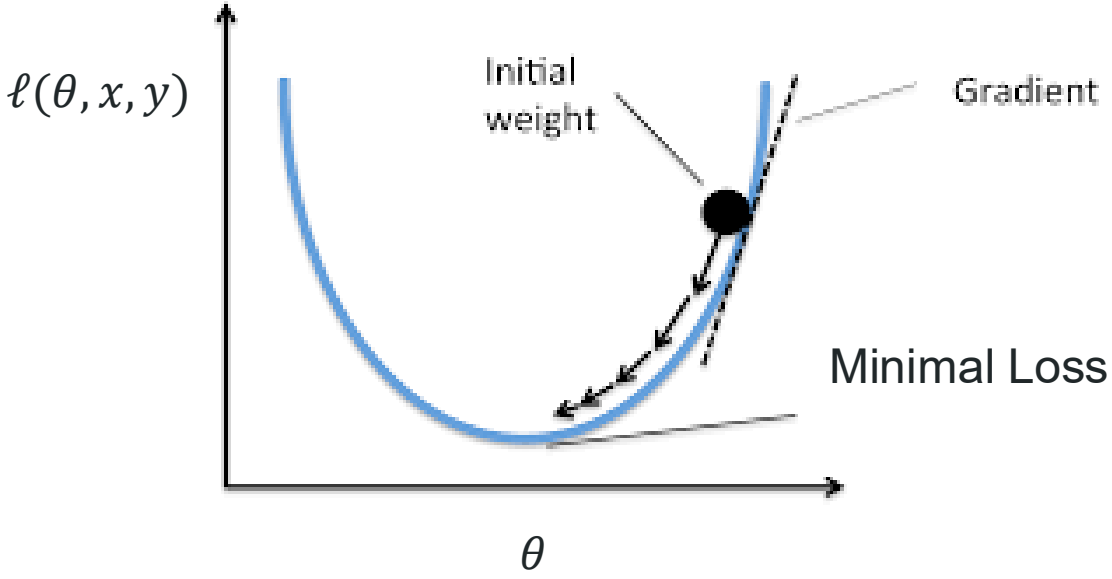
$$\mathcal{L}(\theta, D) = \frac{1}{N}\sum_{i} \ell(\theta, x_i, y_i)$$

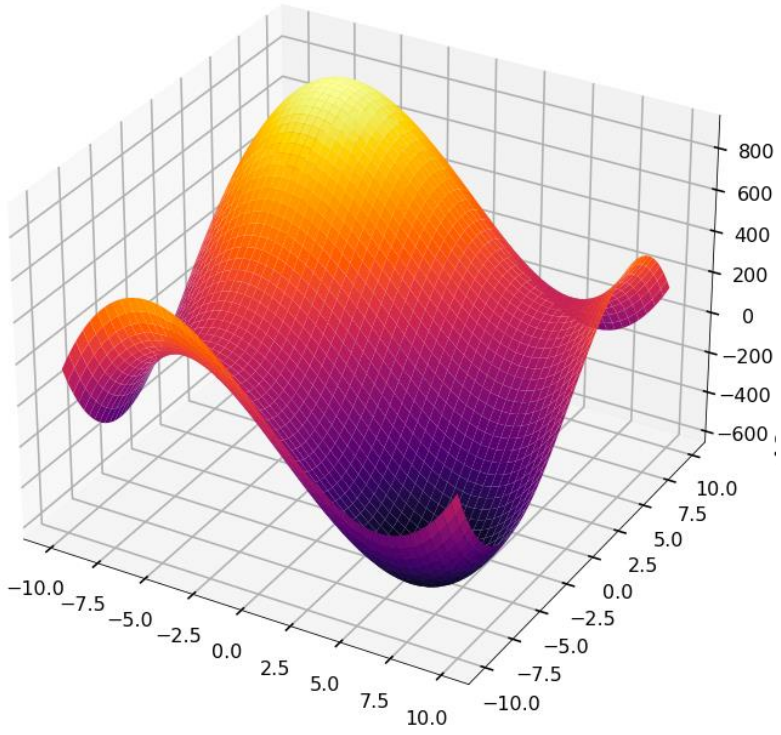
- In practice, the minimization is done using Stochastic Gradient Descent (SGD).



Gradient Descent

- The gradient of the loss $\nabla \ell(\theta, x, y)$ evaluated at sample (x, y) is the derivative with respect to each parameter θ_i (every w and b).
- It tells us the direction in which θ should go to minimize the loss (for sample (x, y)).





Gradient Descent

- We could minimize the loss by running several steps (epochs) of Gradient Descent:
  - For each step $t \in [T]$:

$$\theta_t = \theta_{t-1} - \eta \nabla \mathcal{L}(\theta_{t-1}, D)$$

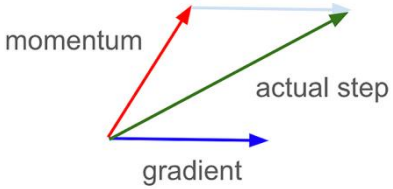
  - η is the *learning rate,*
    - *i.e., how big of a step you take*

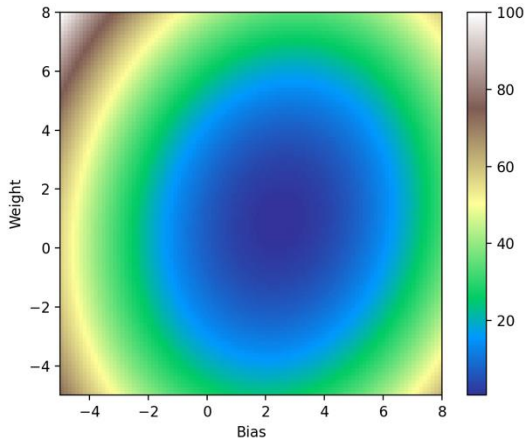
- This is expensive, so usually we do these iterations over a subset of the training sets (**batches**)
- Note that θ represents parameters, η and T are hyper-parameters

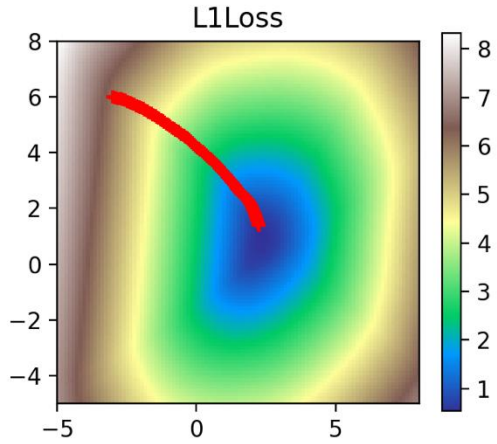
Stochastic Gradient Descent – with Mini Batches

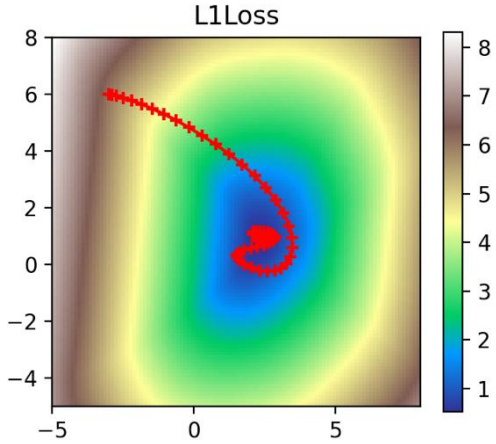
For each training step $t \in [T]$:

1. Take a batch B of L samples from D
2. For each sample $(x_i, y_i) \in B$, compute the gradient $\mathrm{g_i} = \nabla \ell(\theta_{t-1}, x_i, y_i)$
3. Average the gradients $g = \frac{1}{L}\sum_i g_i$
4. Descend $\theta_t = \theta_{t-1} - \eta \cdot g$





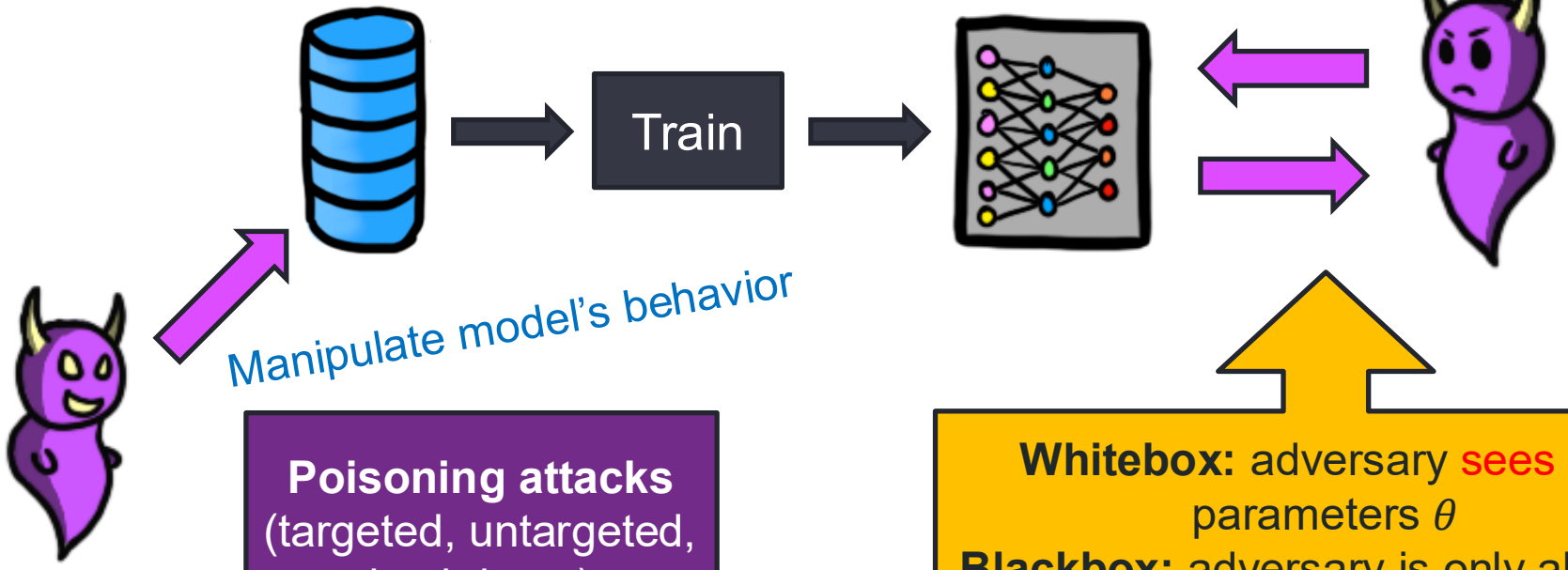




Inference Attacks in ML

Attacking ML models

- There are many types of attacks against ML
- Later we will see that there are also different types of defenses



Stealing training data

Inference Attacks:
- Membership inference
- Attribute inference
- Property inference
- Model inversion

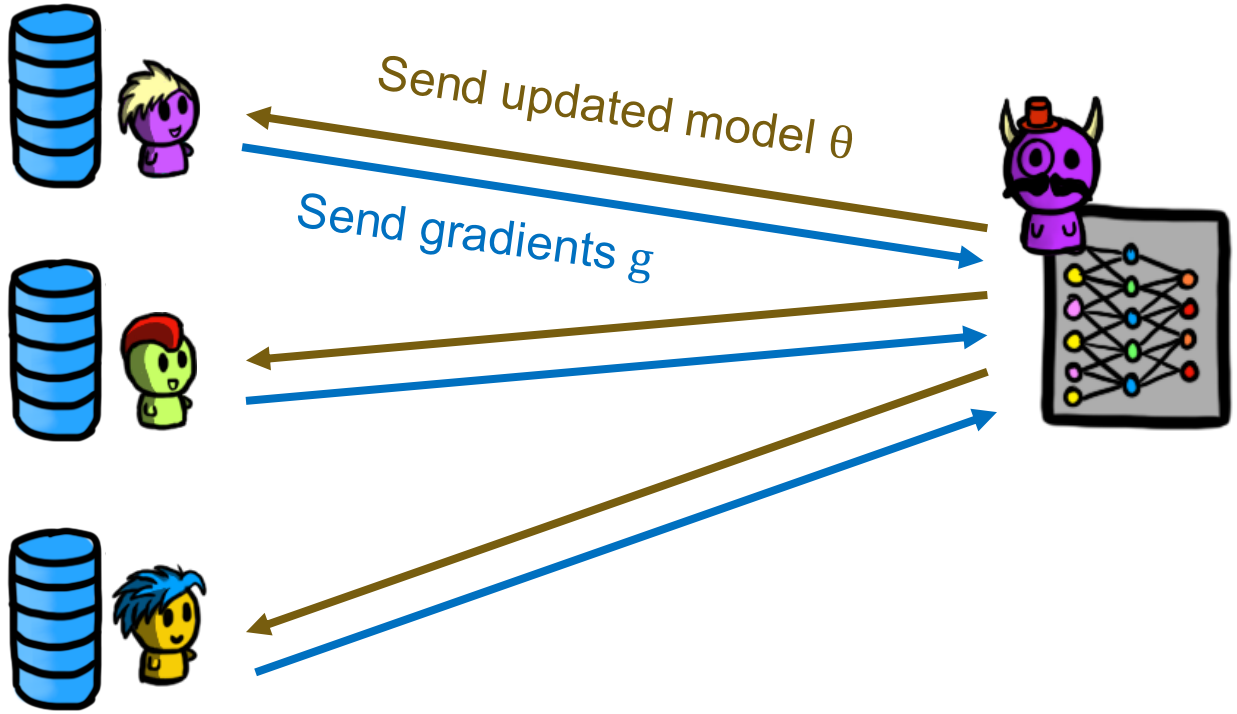
Evasion attacks
Model stealing attacks

Poisoning attacks
(targeted, untargeted, backdoors)

Whitebox: adversary sees the parameters θ
Blackbox: adversary is only allowed to send queries

Attacking ML models in Federated Learning

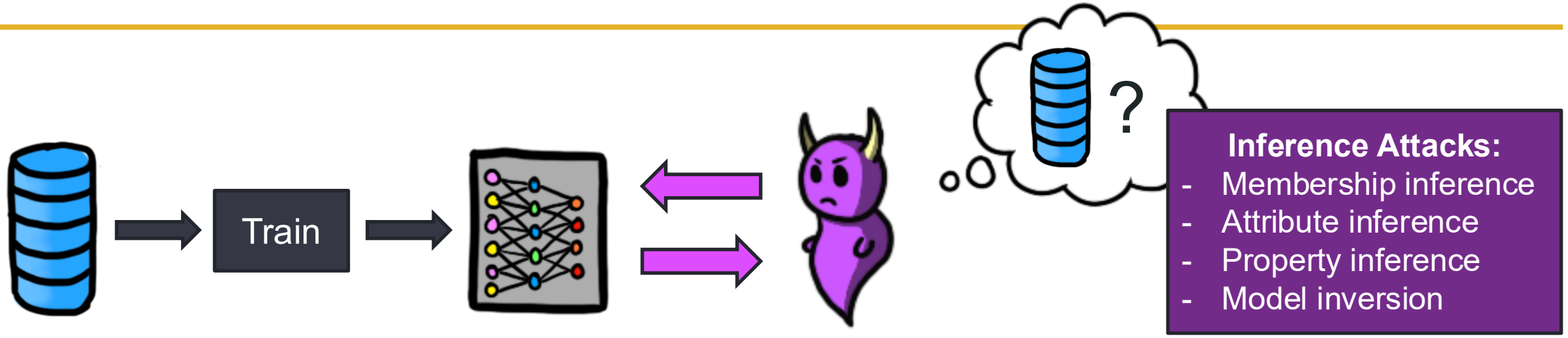
- Federated Learning: a centralized server builds a model, a set of clients send updates (gradients) using their local datasets



Poisoning attacks (targeted, untargeted, backdoors)

Inference Attacks:
(adv sees all intermediate gradients, can potentially send malicious θ)
- Membership inference
- Attribute inference
- Property inference
- …

Inference attacks



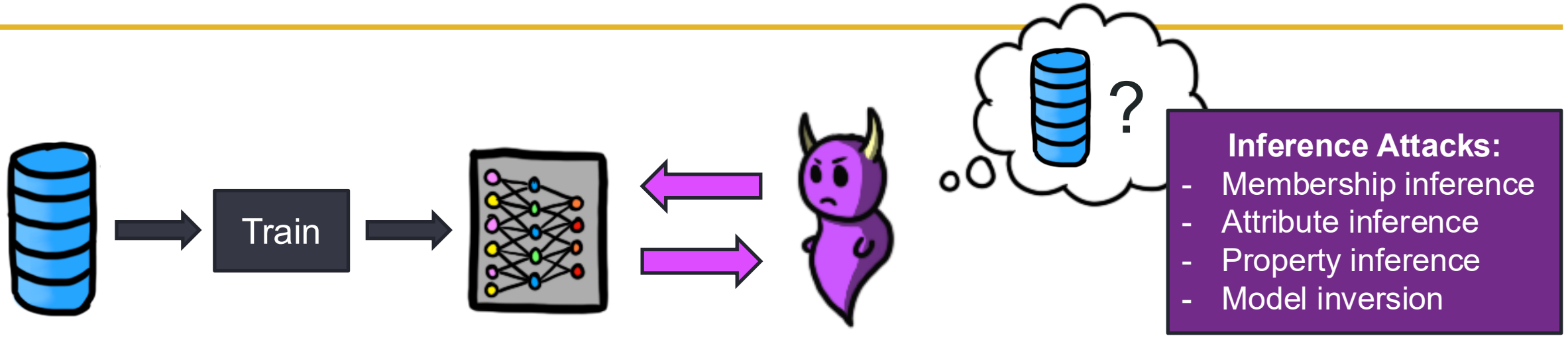
Membership Inference:
Is a given sample in the training set?

Attribute Inference:
Given a sample with some missing attributes, can we guess them?

Property Inference:
Given a property about the *whole* training set, can we guess if it's true or not?

Model inversion:
Given a label, can we find a representative element of this class? (learn x from y)

Inference attacks



Membership Inference:
Is a given sample in the training set?

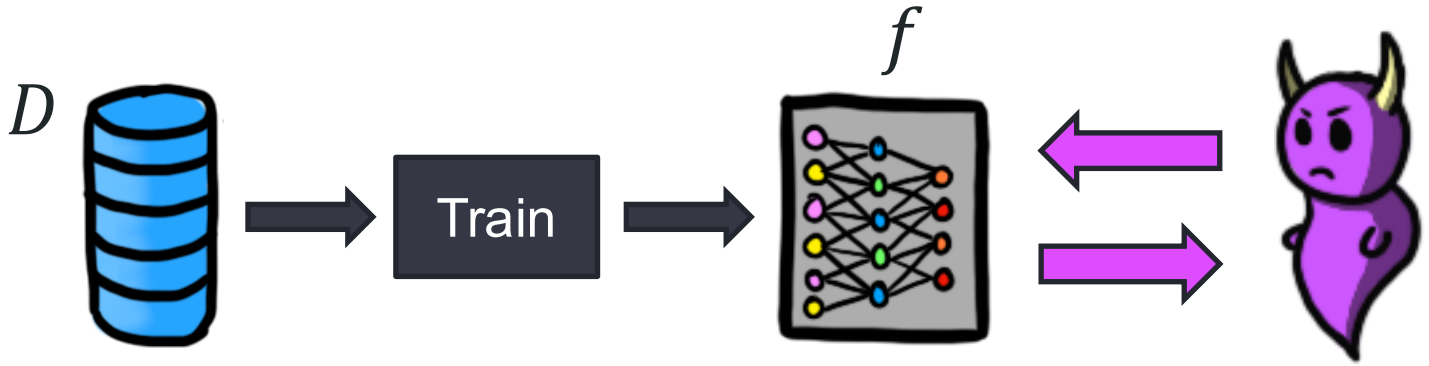
Attribute Inference:
Given a sample with some missing attributes, can we guess them?

Property Inference:
Given a property about the *whole* training set, can we guess if it's true or not?

Model inversion:
Given a label, can we find a representative element of this class? (learn x from y)

Membership Inference Attacks (MIAs)

- Given a sample (x, y), and a model f trained with dataset D, guess whether $(x, y) \in D$.



Black-box: the adversary queries the model (possibly more than once)
White-box: the adversary sees the model parameters θ

- With only black-box access, and a model that outputs confidence scores:
  - $f(x) = [\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_k]$, where $\hat{y}_j$ are confidence scores for label j.

Q: If you were the adversary, with a target sample (x, y) and black-box access to the model f, how would you guess if the target sample is a member?

Threshold Attacks

- *Idea:* the model will be more confident on samples it has seen during training.

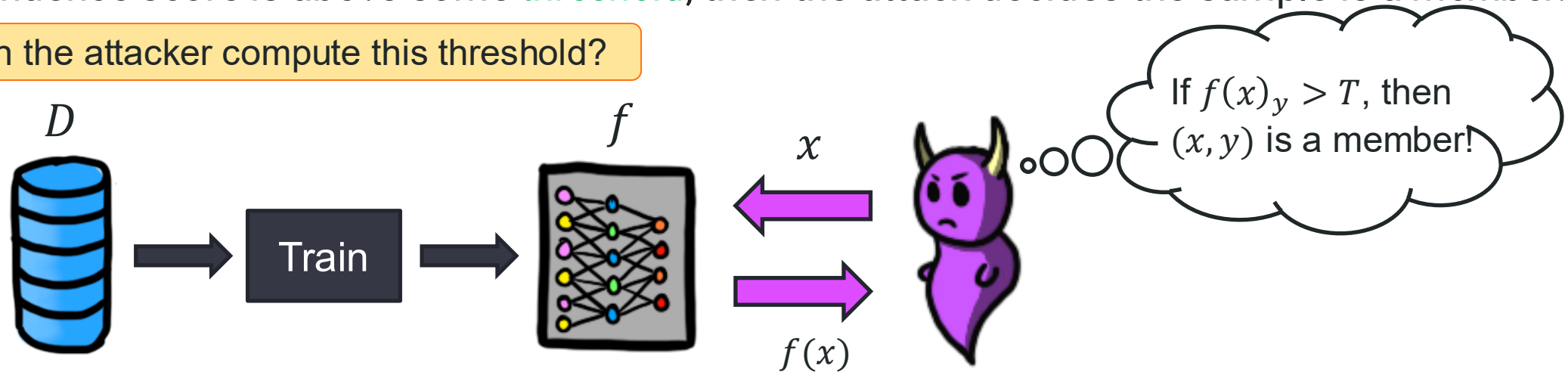
Threshold Attacks

- *Idea:* the model will be more confident on samples it has seen during training.

Threshold attack

- This attack queries the model on sample x and then measures the confidence score assigned to its true label y.
- If the confidence score is above some *threshold*, then the attack decides the sample is a *member*.

Q: how can the attacker compute this threshold?



Yeom et al. "Privacy risk in machine learning: Analyzing the connection to overfitting." *CSF*, 2018.

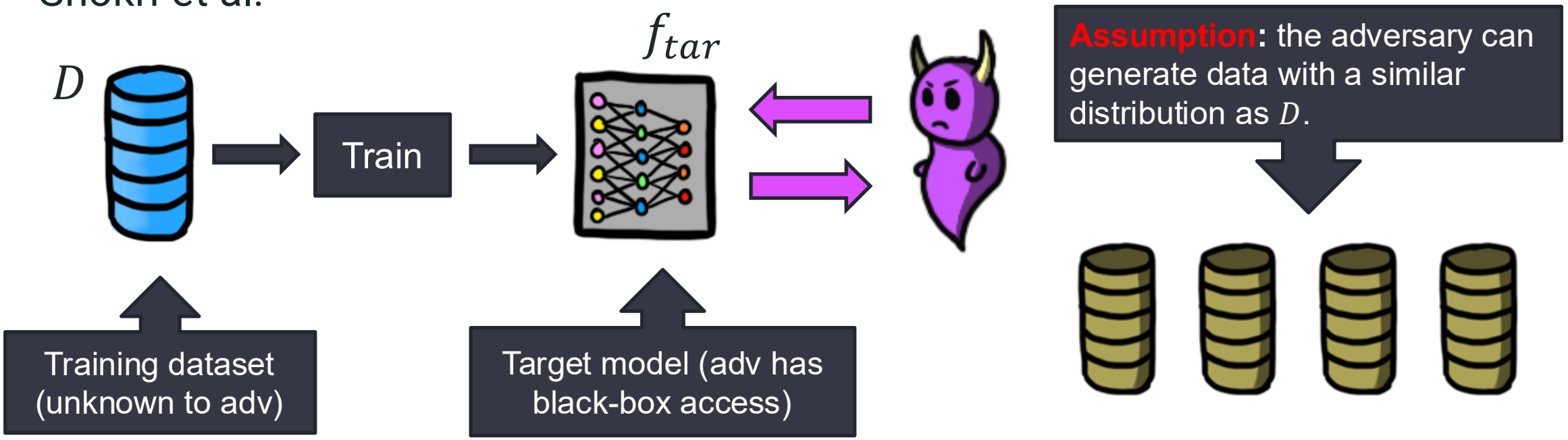
Neural Network-based Attacks

- Other MIAs use Machine Learning against Machine Learning.



Neural Network-based Attacks

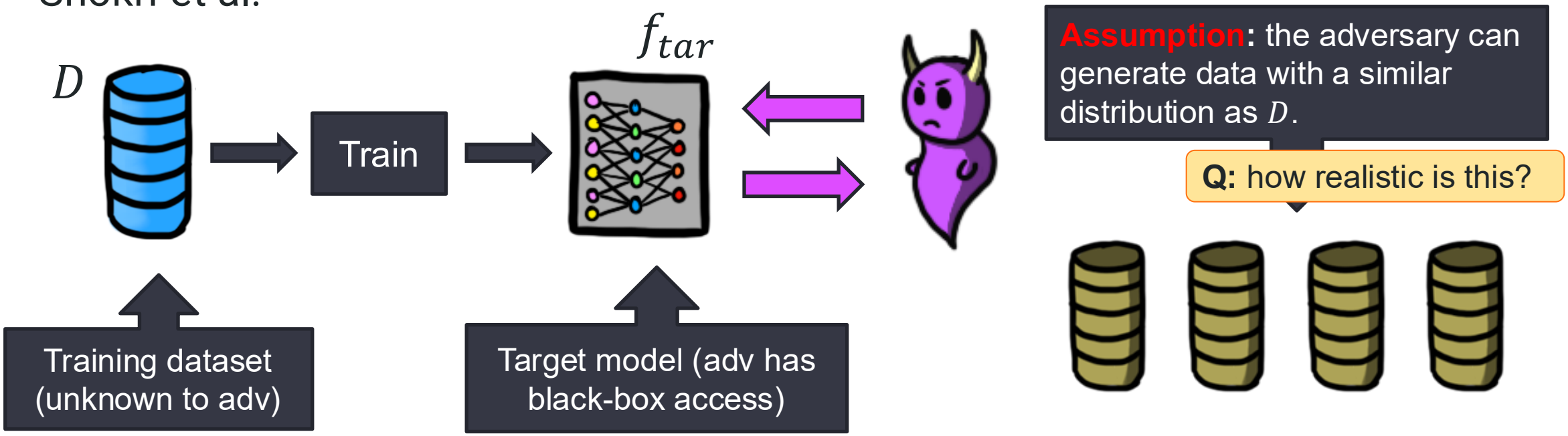
- Other MIAs use Machine Learning against Machine Learning.
- The first NN-based attack (which was also the first MIA) was proposed by Shokri et al.



Shokri, Reza, et al. "Membership inference attacks against machine learning models." *IEEE symposium on security and privacy (SP)*, 2017.

Neural Network-based Attacks

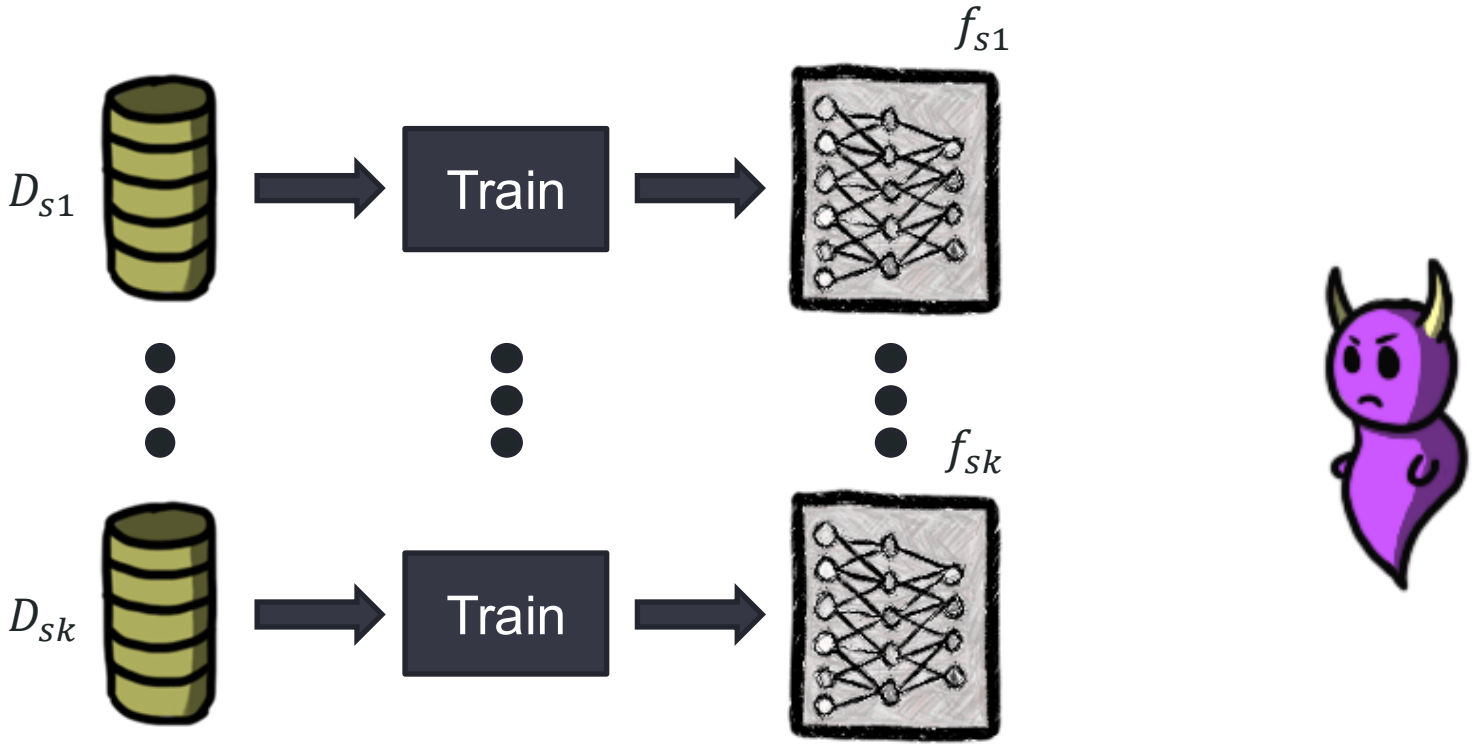
- Other MIAs use Machine Learning against Machine Learning.
- The first NN-based attack (which was also the first MIA) was proposed by Shokri et al.



Shokri, Reza, et al. "Membership inference attacks against machine learning models." *IEEE symposium on security and privacy (SP)*, 2017.

Shokri et al.’s attack

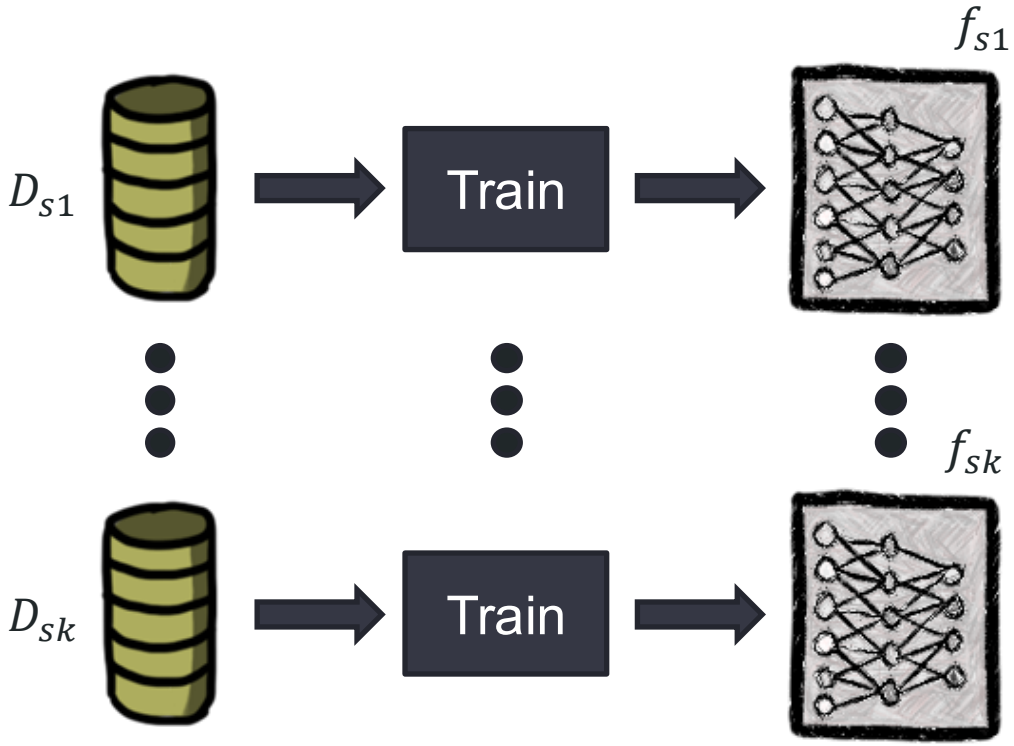
1. Generate **shadow** training datasets $D_{s1}, D_{s2}, \ldots, D_{sk}$ (based on D’ with distribution similar to D).
2. Train k **shadow models** $f_{s1}, \ldots, f_{sk}$ (same classification task as the target model “AWS MLaaS”).



Shokri, Reza, et al. "Membership inference attacks against machine learning models." *IEEE symposium on security and privacy (SP)*, 2017.

Shokri et al.'s attack

1. Generate **shadow** training datasets $D_{s1}, D_{s2}, \ldots, D_{sk}$ (based on D' with distribution similar to D).
2. Train k **shadow models** $f_{s1}, \ldots, f_{sk}$ (same classification task as the target model "AWS MLaaS").



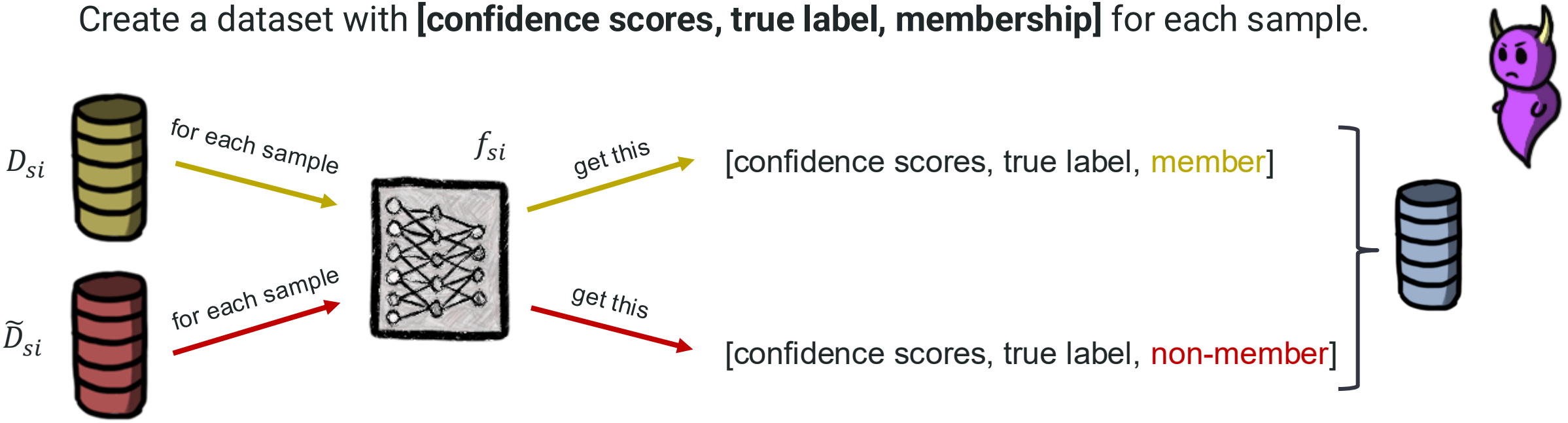


Works even with different model architectures "e.g., classification task for animals"
(but better if you know the actual architecture)

Shokri, Reza, et al. "Membership inference attacks against machine learning models." *IEEE symposium on security and privacy (SP)*, 2017.

Shokri et al.’s attack

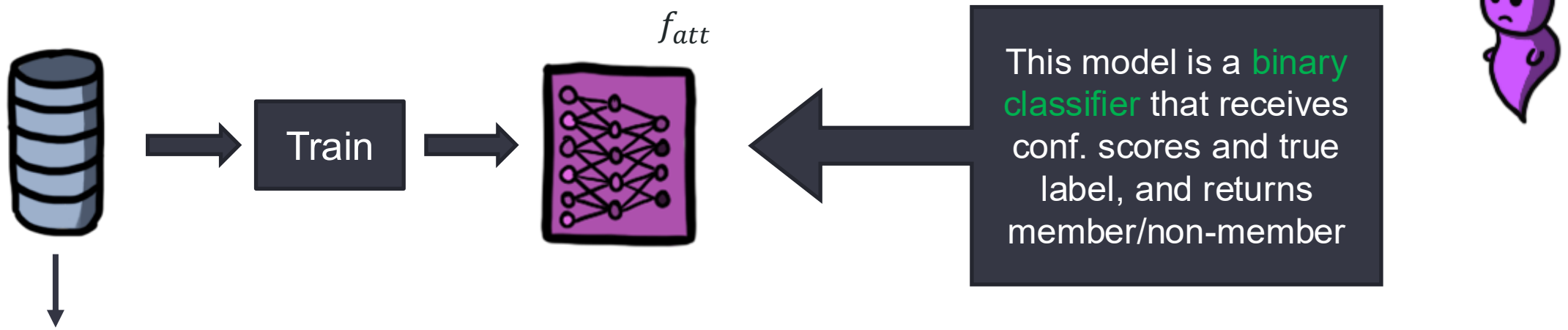
3. Generate **shadow** test data $\widetilde{D}_{s1}, \widetilde{D}_{s2}, \ldots, \widetilde{D}_{sk}$.
4. For each shadow model $i \in [k]$: get the confidence scores for each sample in D_{si} and $\widetilde{D}_{si}$. Create a dataset with **[confidence scores, true label, membership]** for each sample.



Shokri, Reza, et al. "Membership inference attacks against machine learning models." *IEEE symposium on security and privacy (SP)*, 2017.

Shokri et al.’s attack

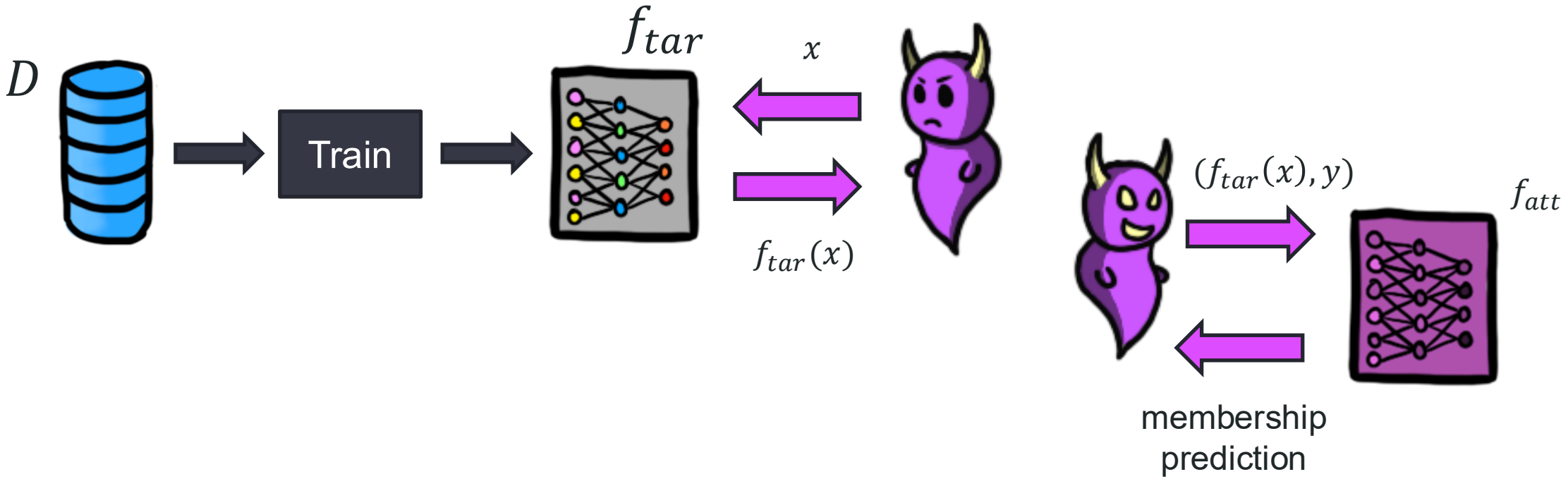
5. With the new dataset, that contains [confidence scores, true label, membership status] computed with all the shadow models, train a new **attack model** f_{att} to predict the **membership status** from **[confidence scores, true label]**



Shokri, Reza, et al. "Membership inference attacks against machine learning models." *IEEE symposium on security and privacy (SP)*, 2017.

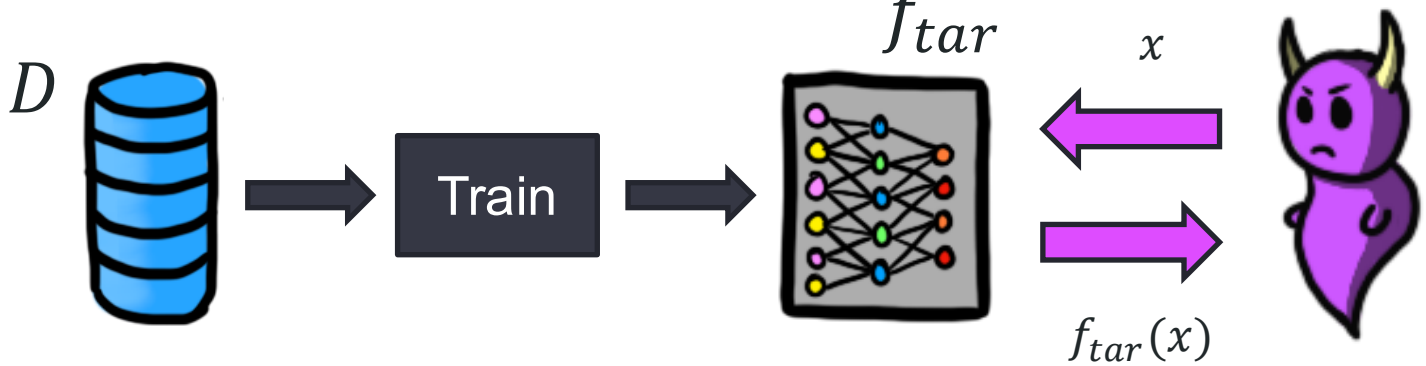
Shokri et al.’s attack

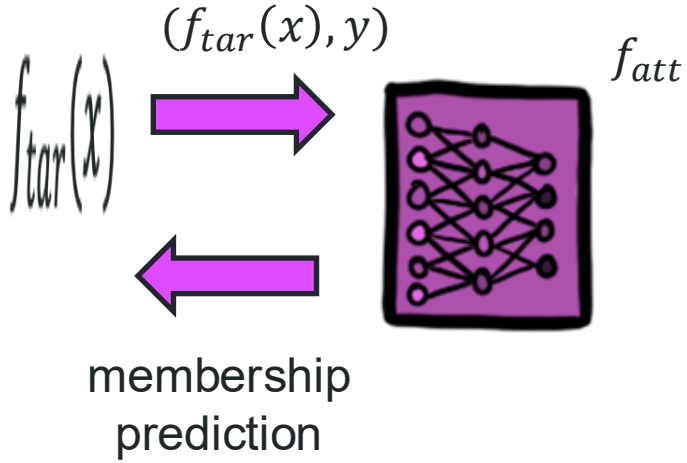
6. Get the confidence scores of the target sample in the target model f_{tar}.
7. Evaluate those **[confidence scores, true label]** samples in the attack model f_{att}.



Shokri, Reza, et al. "Membership inference attacks against machine learning models." *IEEE symposium on security and privacy (SP)*, 2017.

Shokri et al.'s attack



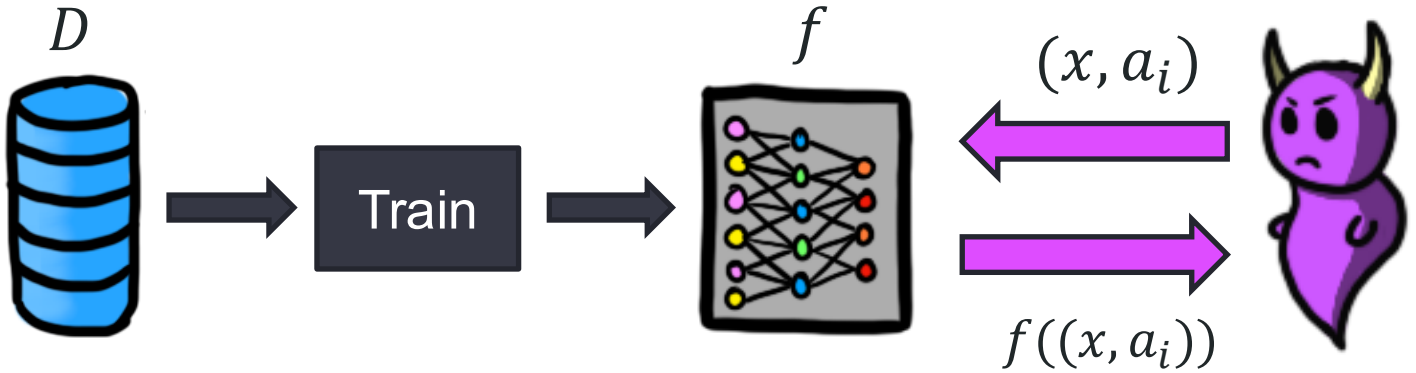


Dataset	*Training Accuracy*	*Testing Accuracy*	*Attack Precision*
Adult	0.848	0.842	0.503
MNIST	0.984	0.928	0.517
Location	1.000	0.673	0.678
Purchase (2)	0.999	0.984	0.505
Purchase (10)	0.999	0.866	0.550
Purchase (20)	1.000	0.781	0.590
Purchase (50)	1.000	0.693	0.860
Purchase (100)	0.999	0.659	0.935
TX hospital stays	0.668	0.517	0.657

The higher the discrepancy between training and testing accuracy
→ The more likely membership inference attack can happen.

Attribute Inference Attacks

- Each sample is $z = (x, a, y)$, where x is the features, a is a privacy-sensitive attribute, and y is the label.
- The adversary has a sample $z = (x, ?, y)$, and wants to learn the attribute.
- Assume the space of all attributes is $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$
- Simple attack: query for all possible samples (x, a_1), …,(x, a_m).
  → The true attribute is probably the one that yields a highest confidence score for the true class y.

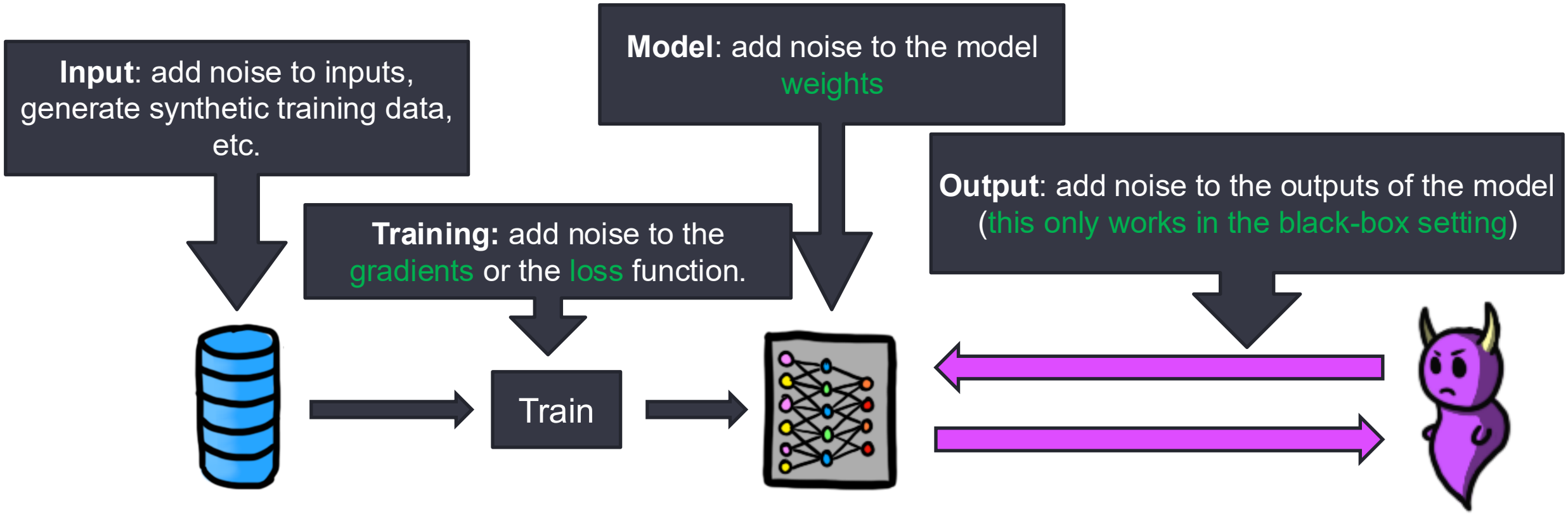


$$\hat{\imath} = argmax_i \, f\big((x, a_i)\big)_y$$

Defenses against inference attacks

Defending against inference attacks

- Where do we defend?



Differentially Private Stochastic Gradient Descent (DP-SGD)

- Adds privacy during the **training** step, modifying SGD.
- **Recall Differential Privacy:** we want to limit the effect that a single sample from the training set has on the output (the "output" of the training algorithm is the "model!")

SGD

For each training step $t \in [T]$:

1. Take a batch B of L samples from D.
2. For each $(x_i, y_i) \in B$, compute the gradient:
$$\mathrm{g_i} = \nabla \ell(\theta_{t-1}, x_i, y_i)$$
3. Average the gradients $g = \frac{1}{L}\sum_i g_i$.
4. Descend $\theta_t = \theta_{t-1} - \eta \cdot g$.

"Private" SGD

For each training step $t \in [T]$:

1. Take a batch B of L samples from D.
2. For each $(x_i, y_i) \in B$, compute the gradient:
$$\mathrm{g_i} = \nabla \ell(\theta_{t-1}, x_i, y_i)$$
3. Average the gradients and add noise $g = \frac{1}{L}(\sum_i g_i + \mathcal{N}(0, \sigma^2))$.
4. Descend $\theta_t = \theta_{t-1} - \eta \cdot g$.

Q: Is it enough to add noise to the gradients?

Differentially Private Stochastic Gradient Descent (DP-SGD)

- The gradient could potentially be **unbounded** → Here, unbounded sensitivity is bad for DP(Algorithm is highly sensitive to individual data points)

e.g., sample with outlier attributes

- We ***clip*** the gradients to ensure their ℓ_2 norm is at most C.
  - C is the *clipping threshold* (1 is usually a good value)
  - C is independent of the data

SGD

For each training step $t \in [T]$:

1. Take a batch B of L samples from D.
2. For each $(x_i, y_i) \in B$, compute the gradient:
$$\mathrm{g_i} = \nabla \ell(\theta_{t-1}, x_i, y_i)$$
3. Average the gradients $g = \frac{1}{L}\sum_i g_i$.
4. Descend $\theta_t = \theta_{t-1} - \eta \cdot g$.

DP-SGD

For each training step $t \in [T]$:

1. Take a batch B of L samples from D.
2. For each $(x_i, y_i) \in B$, compute the gradient:
$$\mathrm{g_i} = \nabla \ell(\theta_{t-1}, x_i, y_i)$$
3. Clip the gradients: $g_i = g_i / \max\left(1, \frac{||g_i||_2}{C}\right)$
4. Sum the gradients $g = \sum_i g_i$.
5. Add noise: $g = g + \mathcal{N}(0, \sigma^2 C^2)$
6. Descend $\theta_t = \theta_{t-1} - \eta \cdot \frac{1}{L} g$.

Differentially Private Stochastic Gradient Descent (DP-SGD)

- The gradient could potentially be **unbounded** → ad for DP
- We ***clip*** the gradients to ensure t norm i
  - C is the *clipping threshold* (1 is us
  - C is independent of the data

NOTE: C should be large enough to avoid significantly impeding the learning process, but small enough to ensure that the gradients are not too sensitive to individual data points.

For each training step t
1. Take a batch
2. For each gradient:
   $$g_i = \nabla \ell(\theta_{t-}$$
3. Average the g nts $g = \frac{1}{L}\sum$
4. Descend $\theta_t = \theta_{t-1} - \eta \cdot g$.

of L samples from D.
$) \in B$, compute the gradient:
$\ell(\theta_{t-1}, x_i, y_i)$

Clip the grad nts: $g_i = g_i / \max\left(1, \frac{||g_i||_2}{C}\right)$

Sum the gradients $g = \sum_i g_i$.

Add noise: $g = g + \mathcal{N}(0, \sigma^2 C^2)$

Descend $\theta_t = \theta_{t-1} - \eta \cdot \frac{1}{L} g$.

DP-SGD: keeping track of ϵ, δ

- Note that a single sample will participate in multiple training steps → there will be some **sequential composition** involved.

- We need to **keep track of ϵ, δ**.
- For a *fixed amount of noise σ*, if we do not keep track of ϵ, δ, we can end up with a very large ϵ, which is bad.
  - The actual *true* ϵ will be smaller than the ϵ we can compute theoretically. – e.g., due to batching, one sample may not appear in a given training step.
  - We can only guarantee an ϵ we can prove w/ theory.

DP-SGD

For each training step $t \in [T]$:

1. Take a batch B of L samples from D.
2. For each $(x_i, y_i) \in B$, compute the gradient:
$$\mathrm{g_i} = \nabla \ell(\theta_{t-1}, x_i, y_i)$$
3. Clip the gradients: $g_i = g_i / \max\left(1, \frac{||g_i||_2}{C}\right)$
4. Sum the gradients $g = \sum_i g_i$.
5. Add noise: $g = g + \mathcal{N}(0, \sigma^2 C^2)$
6. Descend $\theta_t = \theta_{t-1} - \eta \cdot \frac{1}{L} g$.

DP-SGD: keeping track of ϵ, δ

- Each update introduces some privacy loss → track how much ϵ has been used.
- First, we choose a δ. Recall that this should be smaller than $\delta < \frac{1}{N}$.
  - The reason is the following:
    a training algorithm that simply publishes a random training set record would provide $(\epsilon = 0, \delta = 1/N)$-DP.
    However, we know this is not private enough.

DP-SGD

For each training step $t \in [T]$:

1. Take a batch B of L samples from D.
2. For each $(x_i, y_i) \in B$, compute the gradient:
   $$\mathrm{g_i} = \nabla \ell(\theta_{t-1}, x_i, y_i)$$
3. Clip the gradients: $g_i = g_i / \max\left(1, \frac{||g_i||_2}{C}\right)$
4. Sum the gradients $g = \sum_i g_i$.
5. Add noise: $g = g + \mathcal{N}(0, \sigma^2 C^2)$
6. Descend $\theta_t = \theta_{t-1} - \eta \cdot \frac{1}{L} g$.

DP-SGD: keeping track of ϵ, δ

Q: Given δ, σ, C, T, and assuming each sample in D is used *once per training step*, what is the total ϵ we get?

- Use naive composition

DP-SGD

For each training step $t \in [T]$:

1. Take a batch B of L samples from D.
2. For each $(x_i, y_i) \in B$, compute the gradient:

$$g_i = \nabla \ell(\theta_{t-1}, x_i, y_i)$$

3. Clip the gradients: $g_i = g_i / \max\left(1, \frac{||g_i||_2}{C}\right)$
4. Sum the gradients $g = \sum_i g_i$.
5. Add noise: $g = g + \mathcal{N}(0, \sigma^2 C^2)$
6. Descend $\theta_t = \theta_{t-1} - \eta \cdot \frac{1}{L} g$.

$\Delta_2^2 = C^2$: second-order sensitivity

Typically refers to the maximum **L_2 norm** of the gradients, and it is directly related to the **gradient clipping bound**

$f(D) + Y$ is (ϵ, δ)-DP if

$$Y \sim N(0, \sigma^2)$$

$$\sigma^2 = 2 \ln\left(\frac{1.25}{\delta}\right) \Delta_2^2 / \epsilon^2$$

DP-SGD: keeping track of ϵ, δ

Q: Given δ, σ, C, T, and assuming each sample in D is used *once per training step*, what is the total ϵ we get?

- Use naive composition

A: $\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2 \quad \rightarrow \epsilon_{step} = \sqrt{2\ln\left(\frac{1.25}{\delta}\right)C^2/\sigma^2}$ for each step. Then naïve composition gives

$$\epsilon_{total} = T.C/\sigma\sqrt{2\ln\left(\frac{1.25}{\delta}\right)}$$

*Note: this question is very over simplified

DP-SGD

For each training step $t \in [T]$:

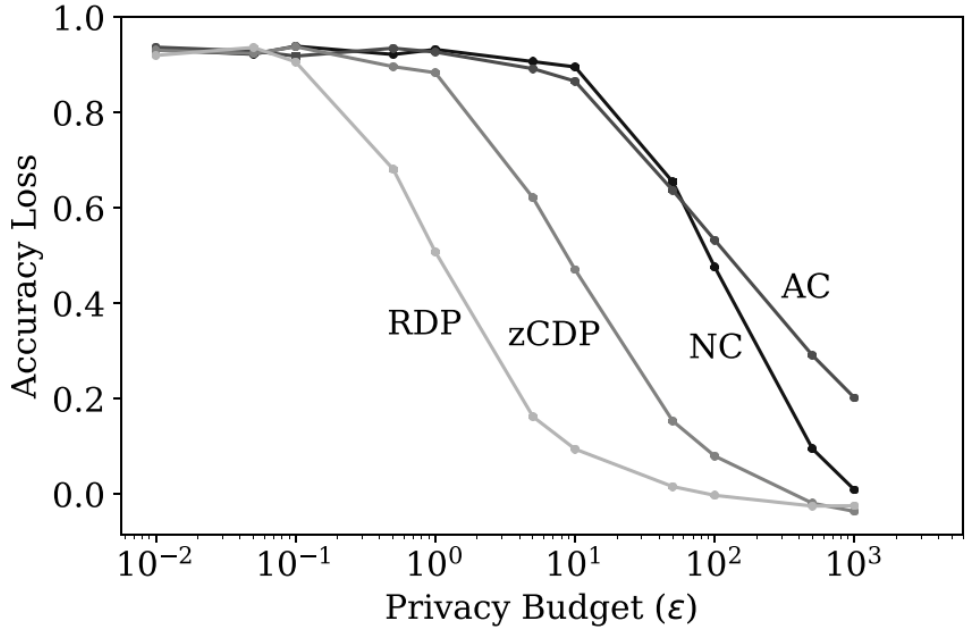
1. Take a batch B of L samples from D.
2. For each $(x_i, y_i) \in B$, compute the gradient:
$$g_i = \nabla\ell(\theta_{t-1}, x_i, y_i)$$
3. Clip the gradients: $g_i = g_i / \max\left(1, \frac{||g_i||_2}{C}\right)$
4. Sum the gradients $g = \sum_i g_i$.
5. Add noise: $g = g + \mathcal{N}(0, \sigma^2 C^2)$
6. Descend $\theta_t = \theta_{t-1} - \eta \cdot \frac{1}{L} g$.

$f(D) + Y$ is (ϵ, δ)-DP if $Y \sim N(0, \sigma^2)$

$$\sigma^2 = 2\ln\left(\frac{1.25}{\delta}\right)\Delta_2^2/\epsilon^2$$

DP-SGD: keeping track of ϵ, δ

- Renyi Differential Privacy (RDP) provides a tighter ϵ, δ bound.
  - Better suited to Gaussian Noise
  - Keeps track of more information
- This means that, for a given σ, C, and δ, RDP tells us our actual ϵ is smaller than what Advanced Composition (AC) tells us.
- In other words, for a target privacy budget ϵ, using RDP we need to add less noise than using AC.
  - E.g., again, because a sample may be excluded from a given training step
- Note that, even with RDP, we need $\epsilon > 100$ if we do not want any accuracy loss

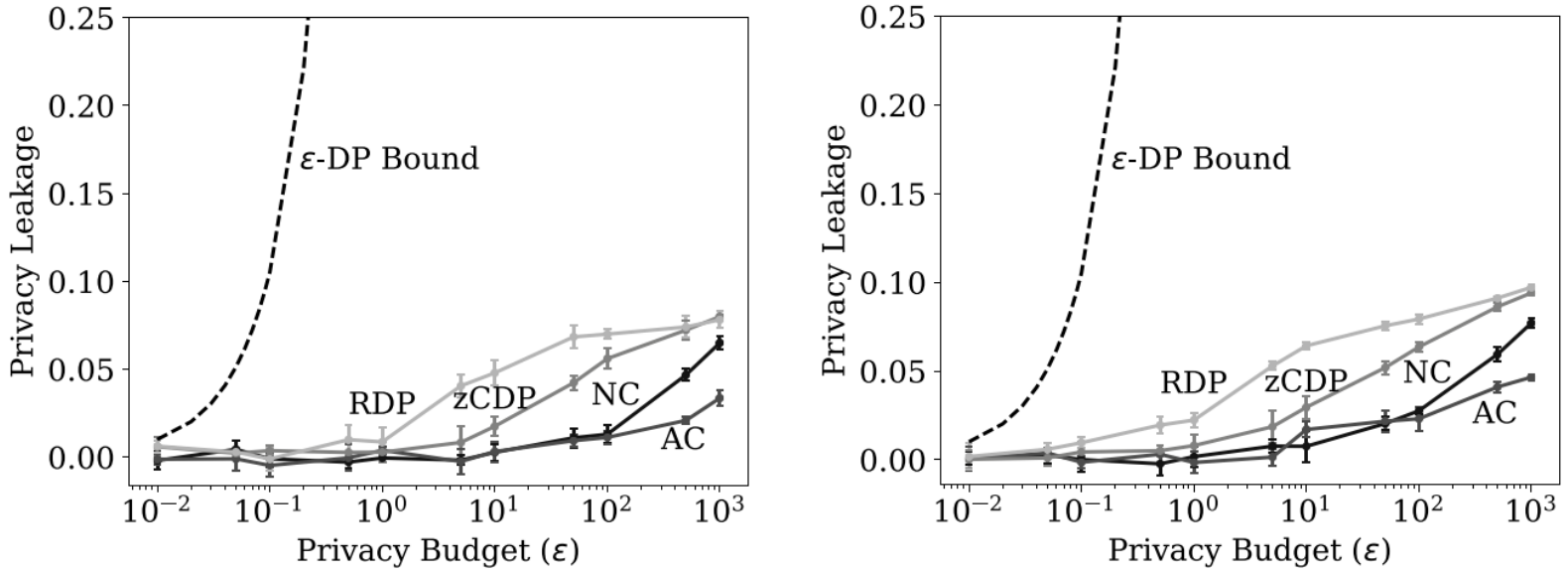


zCDP: Zero-Concentrated Differential Privacy
NC: Noise-Contrastive privacy
AC: Approximate Composition

Jayaraman, Bargav, and David Evans. "Evaluating differentially private machine learning in practice." *USENIX Security Symposium*. 2019.

DP-SGD: theoretical vs empirical privacy

- Both attacks we've seen perform similarly
- It seems that $\epsilon = 100$ or even $\epsilon = 1000$ still provides good empirical privacy
- The theoretical bound "worst-case" on the privacy leakage provided by DP is very loose "overestimates the actual risk".



(a) Shokri et al. membership inference

(b) Yeom et al. membership inference

Jayaraman, Bargav, and David Evans. "Evaluating differentially private machine learning in practice." *USENIX Security Symposium*. 2019.

Issues of DP-SGD

- We saw that, for strong theoretical privacy (e.g., $\epsilon < 1$), the models usually lose all utility.
- For very weak theoretical privacy (e.g., $\epsilon = 100$), some models achieve reasonable utility.
- However, DP-SGD with $\epsilon = 100$ seems to provide enough protection against existing attacks.

Q: Is it OK to use $\epsilon = 100$?

Issues of DP-SGD

- We saw that, for strong theoretical privacy (e.g., $\epsilon < 1$), the models usually lose all utility.
- For very weak theoretical privacy (e.g., $\epsilon = 100$), some models achieve reasonable utility.
- However, DP-SGD with $\epsilon = 100$ seems to provide enough protection against existing attacks.

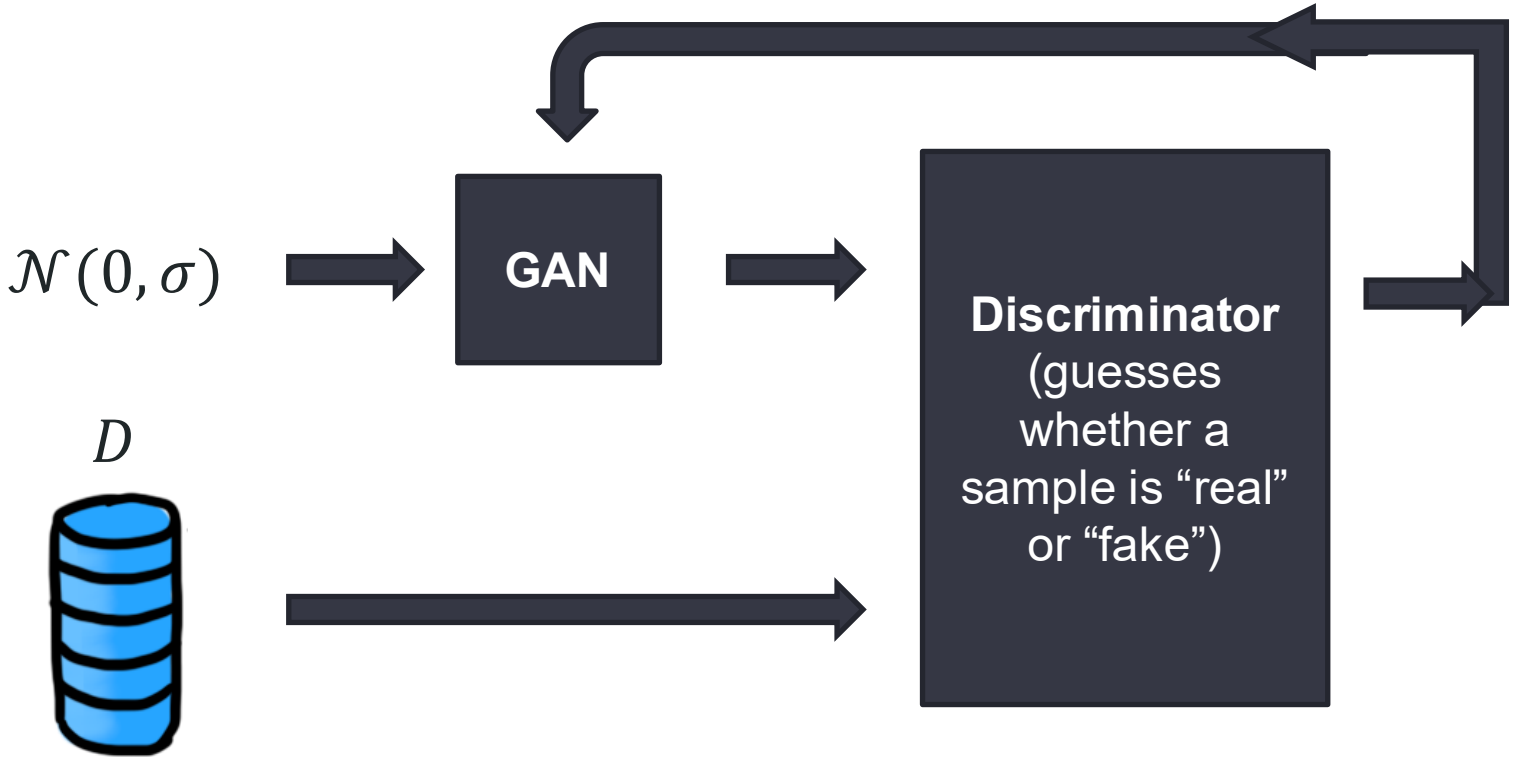
Q: Is it OK to use $\epsilon = 100$?

A: It might be OK to use DP-SGD tuned to $\epsilon = 100$, but at that point we might as well use defenses that do not provide DP, since the DP guarantee is already meaningless at that point.

More defenses…

Synthetic Data Generation

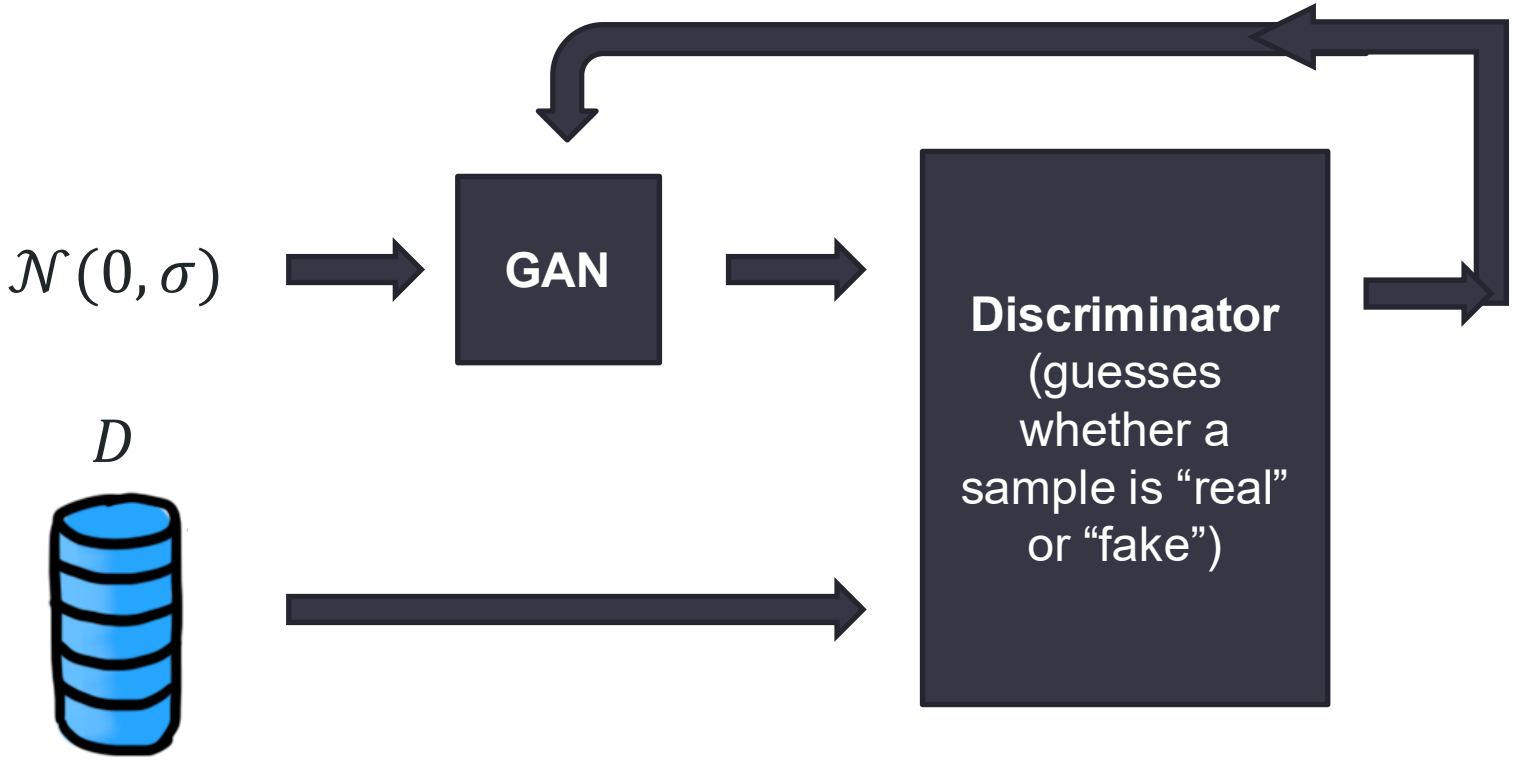
- For example, by using a GAN to generate real-looking synthetic samples:



If we train the GAN using privacy-preserving training algorithms (e.g., DP-SGD on the discriminator), we can use it to generate a privacy-preserving synthetic dataset!

Synthetic Data Generation

- For example, by using a GAN to generate real-looking synthetic samples:

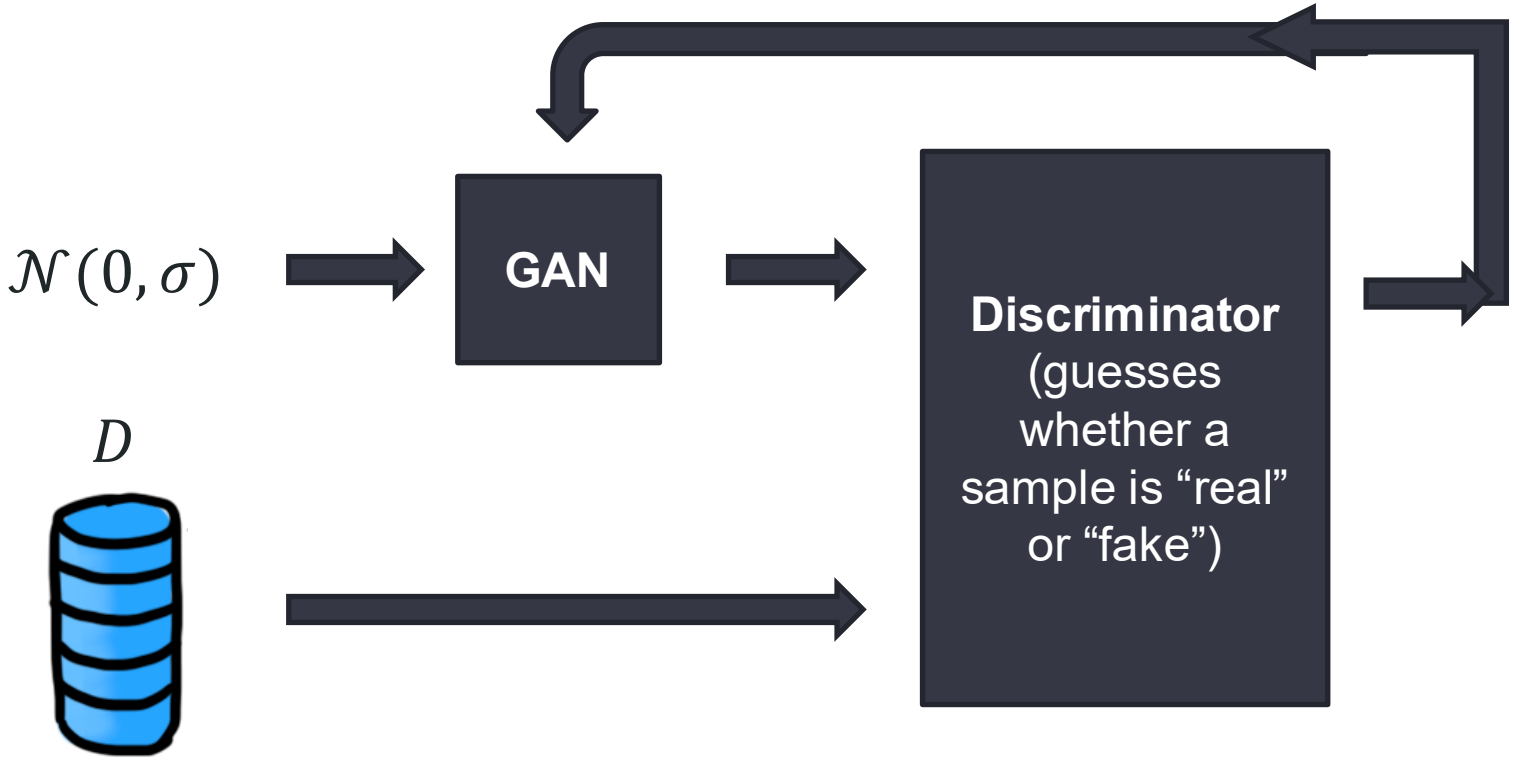


If we train the GAN using privacy-preserving training algorithms (e.g., DP-SGD on the discriminator), we can use it to generate a privacy-preserving synthetic dataset!

Q: What can we do with the resulting dataset?

Synthetic Data Generation

- For example, by using a GAN to generate real-looking synthetic samples:



If we train the GAN using privacy-preserving training algorithms (e.g., DP-SGD on the discriminator), we can use it to generate a privacy-preserving synthetic dataset!

Q: What can we do with the resulting dataset?

A: Anything!

Other defenses

- There are defenses that add **noise** to the confidence scores (MemGuard [Jia et al.]), but are not very effective.
- MIAs can work even if the model just leaks the predicted label (and not the confidence scores)
- Sometimes, **generalization** is a good defense by itself:
  - A well-generalized model will perform similarly in members (training set) and non-members (testing set)
  - Therefore, it will be harder for an adversary to decide whether a sample is a member or non-member if the model generalizes well.
  - Generalization is also **good for utility** (improves test accuracy), so it's a win-win.